

STUDY GUIDE · SERIES 2 OF THE DISTRIBUTED SYSTEMS TRACK

Distributed Systems: Consensus

Raft & Paxos — A Beginner-to-Advanced Study Guide

Prepared 2026-06-21 · Read the sections in order

Table of Contents

How to Use This Guide

CORE CONCEPTS

1. The Consensus Problem

2. Replicated State Machines & the Log

3. Raft — Leader Election

4. Raft — Log Replication, Safety & Membership

5. Paxos — The Original Consensus Algorithm

6. Multi-Paxos, Raft vs Paxos & the Real World

REFERENCE

Glossary of Terms

Frequently Asked Questions

Revision Cheat Sheet

How to Use This Guide

Consensus is the problem of getting a group of separate computers to agree on one value — one decision, one order of events — even though some of them may crash, restart, or fall out of touch at the worst possible moment. It sounds small. It is the single hardest problem in distributed systems, because the network can lose, delay, or reorder messages, and a machine that has gone quiet is indistinguishable from one that is merely slow. Solve consensus reliably and you can build a system that stays correct and available while its parts fail underneath you. That is why consensus is the crown jewel of the field: databases, lock services, configuration stores, and schedulers all rest on it.

This is **Series 2: Consensus**. It assumes you have read Series 1 (Foundations) and are comfortable with the basics: nodes, messages, latency, replication, failure models, and why "just use one server" eventually breaks. We build on that — we do not re-teach it.

The six sections are deliberately ordered so each one earns the next:

- **1. The Consensus Problem** — what agreement actually requires (agreement, validity, termination), why a majority is the magic number, and why FLP says no algorithm can be perfect.
- **2. Replicated State Machines & the Log** — the universal trick: if every node applies the *same commands in the same order*, they stay identical. Consensus reduces to agreeing on a log.
- **3. Raft — Leader Election** — Raft's first job: pick one leader per term so there is a single source of truth for the log.
- **4. Raft — Log Replication, Safety & Membership** — how the leader copies entries, commits them by majority, guarantees safety, and changes the cluster's membership without breaking.
- **5. Paxos — The Original Consensus Algorithm** — the foundational algorithm Raft was reacting to: proposers, acceptors, and the two-phase prepare/accept dance.
- **6. Multi-Paxos, Raft vs Paxos & the Real World** — running Paxos for a stream of decisions, an honest comparison, and the systems (etcd, ZooKeeper, Spanner) that ship this for real.

Read in order on a first pass. The narrative is "problem → the log abstraction → Raft (taught as election then replication) → Paxos → the real world." If you only want a working mental model, read 1–4 (Raft) and skim 5–6. Use the Glossary as you go, the FAQ when something feels contradictory, and the Cheat Sheet to revise before an interview or a design review.

Throughout, look for **key** boxes (the one idea you must keep), **tips** (practical guidance), and **warnings** (the traps that bite real engineers).

1. The Consensus Problem

This whole guide is about one deceptively small word: **agree**. You already know that in a distributed system there are many machines (we call them *nodes*), the network can lose, delay, duplicate, or reorder messages, and any node can crash at any moment without warning. Now we ask a brutal question: given all that chaos, how do we get a group of machines to **agree on a single value** and never go back on it? That question is the **consensus problem**, and solving it correctly is the hardest and most important thing in distributed systems. Raft and Paxos — the algorithms this guide teaches — exist for exactly one reason: to solve consensus. So before we touch either of them, we need to understand the problem they are solving, why it is so hard, and what "correct" even means.

1.1 What "consensus" actually means

Consensus is the act of getting a group of nodes to all decide on the *same single value*, even though some of those nodes may crash and some messages may be lost or delayed. The value can be anything: a number, the name of a leader, the next operation to run, "yes commit this transaction", or "node 7 holds the lock now". The key word is *single* — everyone who decides must decide on the *same* thing, and once a node has decided, it can never change its mind.

Let's be precise about the setup, because the precision is what makes it hard:

- Several nodes each start with a **proposed value** (an input they would like the group to choose). For a leader election, each node might propose "I should be leader" or "node 3 should be leader".
- The nodes exchange messages over an unreliable network.
- At the end, each non-crashed node outputs a **decided value**.
- The protocol is *correct* only if all those decided values are identical, the decided value was one that somebody actually proposed, and the nodes eventually reach a decision rather than arguing forever.

If that sounds easy, remember: you cannot tell a crashed node from a slow one (no global clock, unreliable network), messages can arrive out of order or never arrive, and any node — including the one currently "in charge" — can die mid-sentence. You have to be correct anyway.

Analogy: Imagine a group of friends planning where to eat dinner, but they can only communicate by passing paper notes through an unreliable courier. Some notes get lost. Some friends leave the room without telling anyone (crash). Some friends are just slow to reply — and from the outside you *cannot tell* "left the room" apart from "still thinking". The challenge is to make sure that when the night ends, *everyone who is still around went to the exact same restaurant*, that the restaurant was one somebody actually suggested, and that they didn't stand around forever unable to decide. That is consensus.

1.2 Why we need consensus — concrete uses

Consensus is not an academic toy. Almost every reliable distributed system has a consensus algorithm beating at its core. Here are the jobs it does:

- **Electing one leader.** Many systems want exactly *one* node "in charge" at a time (to avoid two nodes giving conflicting orders). The cluster must agree on *who* the leader is. If two nodes each believe they are the sole leader — a **split brain** — you get data corruption. Consensus prevents that.
- **Agreeing on the order of operations.** If five replicas of a database each apply writes in a different order, they end up with different data. Consensus lets the cluster agree on *one* ordered log of operations, so every replica applies the same operations in the same order and ends up identical. This is the heart of *replicated state machines*, which we cover later.
- **Committing a transaction.** "Did we all agree to commit, or all agree to abort?" Every participant must reach the *same* verdict — you cannot have one node commit while another aborts.
- **Agreeing who holds a lock.** A distributed lock is just consensus on the question "which client currently owns this lock?" If two clients think they hold the same lock, they can both edit the same resource and corrupt it.
- **Agreeing on cluster membership and configuration.** Which nodes are part of the cluster right now? What is the current config? When you add or remove a server, every node must agree on the new membership at the same logical moment, or messages get sent to ghosts and quorums get miscounted.

Example: `etcd` (the database behind Kubernetes) stores critical cluster state — which pods exist, what the desired configuration is. It runs the **Raft** consensus algorithm internally. When you run `kubectl apply`, the change is proposed to the Raft cluster; a majority of `etcd` nodes must agree to append it to the shared log before it is considered committed. That is why Kubernetes can survive a node dying without losing or corrupting its cluster state — consensus guarantees every surviving `etcd` node has the exact same data in the exact same order.

1.3 The formal requirements — in plain words

A protocol "solves consensus" only if it satisfies three properties. These are the rules of the game. Memorize them; everything Raft and Paxos do is in service of these three.

Property	Plain-English meaning	What breaks if you violate it
Agreement	No two non-faulty nodes decide on <i>different</i> values. If I decided "X", nobody else ever decides "Y".	Split brain, divergent replicas, two leaders, corrupted data.
Validity (also called Integrity)	The value that gets decided must be one that some node actually <i>proposed</i> . The system cannot invent a value out of thin air, and a node decides at most once.	The cluster "agrees" on a value nobody asked for — agreement on garbage is useless.
Termination (also called Liveness)	Every non-faulty node <i>eventually</i> decides something. The protocol does not get stuck forever.	The cluster argues forever and never makes progress; the system hangs.

Notice the tension. **Agreement** and **Validity** are about *never being wrong*. **Termination** is about *eventually making progress*. A protocol that does nothing at all is perfectly safe (it never decides anything wrong) but useless (it never decides anything at all). A protocol that decides instantly always terminates but might let two nodes pick different values. The art of consensus is satisfying all three at once, and as we will see, in the worst case you literally cannot guarantee all three.

Common mistake: Thinking *Agreement* just means "the nodes vote and majority wins". Agreement is much stronger: *once any node has decided, that decision is final and no other node may ever decide differently — not even after a crash, a network partition, a restart, or a leader change*. The decision must survive failures. That permanence is what makes it hard.

1.4 Safety vs liveness — the two halves of "correct"

Distributed-systems people split "correctness" into two categories, and you will hear these words constantly:

- **Safety** = "*never do a wrong thing*." A safety property says some bad thing *never* happens. **Agreement** and **Validity** are safety properties: we never let two nodes decide differently, and we never invent a value. If a safety property is violated even once, the damage is permanent — you cannot un-corrupt the data.
- **Liveness** = "*eventually do a good thing*." A liveness property says some good thing *eventually* happens. **Termination** is a liveness property: eventually a decision gets made. Violating liveness means the system is slow or stuck, which is bad — but it is recoverable. When the network heals, it can make progress again.

The crucial engineering insight: **good consensus algorithms never sacrifice safety, only liveness**. When the network is broken or too many nodes are down, Raft and Paxos will *refuse to make progress* (they pause, breaking liveness temporarily) rather than risk deciding two different values (breaking safety forever). A stalled cluster is annoying; a corrupted cluster is a disaster. They always pick "annoying".

Analogy: Safety is "the bank never lets your balance go negative or double-spends your money." Liveness is "the ATM eventually gives you your cash." If the network to the bank is down, a well-designed ATM says "sorry, try later" (liveness fails, temporarily) rather than handing out money it can't verify (safety fails, permanently). You would much rather wait than have your account corrupted. Consensus algorithms make the same choice.

1.5 The FLP impossibility result — why this is provably hard

Here is the bombshell that shaped the entire field. In 1985, three researchers — Michael Fischer, Nancy Lynch, and Mike Paterson — proved a theorem now known as **FLP impossibility**. We will only build the intuition, not the formal proof, but the intuition is what matters.

First, two definitions:

- A **fully asynchronous system** is one where there is *no upper bound* on how long a message can take to arrive and no upper bound on how long a node can take to do a step. Messages always arrive eventually, but "eventually" could be one millisecond or one million years — you are never allowed to assume a limit.
- A **deterministic** algorithm is one with no randomness: given the same inputs and messages, it always does the same thing.

The FLP theorem states:

In a fully asynchronous system, there is no deterministic algorithm that solves consensus and is guaranteed to terminate, if even a single node may crash.

Read that again. Even with **just one** possible crash, and even though messages are never actually lost (just arbitrarily delayed), you *cannot* build a deterministic protocol that always satisfies all three properties. Some execution will always exist where the protocol runs forever without deciding.

The intuition for why. The root cause is the thing you already know from the foundations guide: in an asynchronous system *you cannot tell a crashed node from a slow node*. When you are waiting for a message from node 3, and it has not arrived, you face an impossible choice:

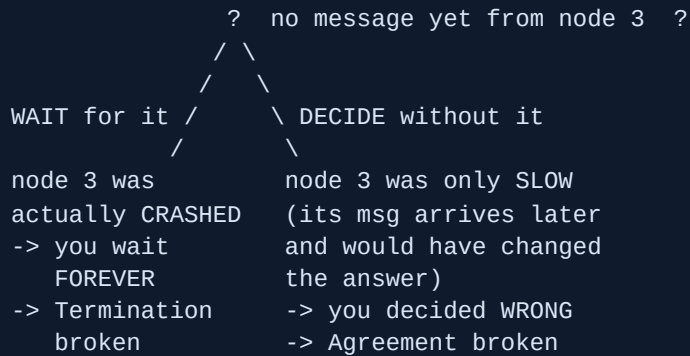
- If you **wait** for node 3, and node 3 has actually crashed, you wait forever — that violates Termination.
- If you **stop waiting** and decide without node 3, and node 3 was merely slow (not crashed) and was about to send a message that would have changed the outcome — you might decide the wrong value, violating Agreement.

The proof formalizes this with a clever argument: it shows there is always a configuration where the decision could still go either way (the experts call this a **bivalent** state — "two-valued", meaning both outcomes are still possible). An adversary who controls only the *timing of messages* (which it is allowed to do, since timing is unbounded) can always delay exactly the one message that would tip the system

toward a decision, keeping it perpetually undecided. The system never crashes, never loses a message — it just stays forever on the fence.

The impossible dilemma at the heart of FLP

You are node A, waiting to hear from node 3 before deciding.



You cannot tell which branch you are in.

No global clock + unbounded delays = no safe choice.

Common mistake: Reading FLP as "consensus is impossible, so Raft and Paxos are lying." Not true. FLP says you cannot *guarantee termination in every possible execution* of a *fully asynchronous, deterministic* system. It does *not* say you can't be correct. Raft and Paxos are always **safe** (Agreement and Validity hold in *every* execution, no exceptions). They only give up the impossible *guarantee* of termination — and in practice they terminate essentially always. FLP constrains the guarantee, not the usefulness.

1.6 How real systems escape FLP

FLP describes a worst case that real networks almost never hit. Real systems sidestep it by adding a pinch of assumption that the pure-asynchronous model forbids. There are three classic escape hatches:

Escape hatch	What it adds	How it dodges FLP	Used by
Timeouts / partial synchrony	Assume that <i>after some unknown point</i> , the network behaves "well enough" — messages arrive within some bound. Use a clock-based timeout to <i>suspect</i> a node has crashed.	A timeout lets you stop waiting for a (probably) dead node and make progress. You might occasionally be wrong (a slow node looks dead), but the algorithm stays safe — it just retries. This restores liveness during good periods.	Raft, Multi-Paxos, ZAB, etcd, Consul
Randomization	Let nodes flip coins so the algorithm is not deterministic.	FLP only forbids <i>deterministic</i> protocols. With randomness, you can guarantee termination with probability 1 (it terminates "almost surely"), dodging the theorem's deterministic assumption.	Randomized Byzantine protocols; Raft uses randomized <i>election timeouts</i> to break ties.
Failure detectors	Add an oracle that gives <i>hints</i> about which nodes have crashed (it may be wrong sometimes).	Even an imperfect failure detector — one that's eventually accurate — provides just enough extra information to make consensus solvable.	The theory underpinning all of the above.

The most important one in practice is **partial synchrony via timeouts**. Raft is built entirely on it: a node waits for a heartbeat from the leader, and if no heartbeat arrives before its *election timeout* fires, it assumes the leader is dead and starts a new election. The timeout is the practical answer to "is it crashed or just slow?" — after waiting long enough, treat it as crashed and move on. If you were wrong, no harm to safety; the system just sorts it out.

Key takeaway: FLP is not a wall that stops us; it is a signpost telling us where the danger is. It says: "You can't have guaranteed termination *and* guaranteed safety in a purely asynchronous world." Real systems respond: "Fine — we keep safety *absolute*, and we buy back liveness with timeouts and randomness, accepting that in a truly pathological network we might pause." Every consensus algorithm you will learn is a different, careful way of making that trade.

1.7 Quorums and majorities — why "more than half" is magic

Now the single most important mechanical idea in all of consensus. You already met *quorums* at a basic level; here is why they make consensus *correct*.

A **quorum** is the minimum number of nodes that must agree before a decision counts. The quorum that consensus algorithms use is a **majority**: *strictly more than half* of all nodes. In a cluster of **N** nodes, a majority is `floor(N/2) + 1` nodes.

Cluster size N	Majority needed	Failures tolerated (N - majority)
3	2	1
5	3	2
7	4	3

Why a *majority* specifically, and not, say, "any 2 nodes"? Because of one beautiful, simple fact:

Any two majorities of the same set must share at least one common member.

This is just counting. If each of two groups contains *more than half* the nodes, then together they contain *more than all* the nodes — which is impossible unless they overlap. So they *must* have at least one node in common. That one shared node is the secret to safety.

```
Cluster of 5 nodes:  [ n1  n2  n3  n4  n5 ]
A majority = 3 nodes.
```

```
Majority A (votes for value X):  n1  n2  n3
Majority B (votes for value Y):           n3  n4  n5
                                         ^^
                                   n3 is in BOTH groups.
```

```
n3 can only have voted for ONE value.
So A and B cannot both succeed with different values.
=> conflicting decisions are mathematically impossible.
```

```
Two groups of "more than half" of 5  ->  3 + 3 = 6 > 5
-> the pigeonhole forces an overlap.
```

Here is why that overlap guarantees **Agreement**. Suppose value X was decided because a majority of nodes accepted it. Later, some other process tries to get value Y decided — it too needs a majority to accept Y. But the X-majority and the Y-majority must overlap in at least one node. That overlapping node already accepted X. The protocol's rules forbid it from also accepting a conflicting Y (each node accepts at most one conflicting value, or carries forward the value it already saw). So Y can never gather a clean majority for a *different* value. The first decision is locked in. **The overlap is the lock.**

This same overlap is also what lets a *new leader* safely learn what the old leader committed. When a new leader is elected by a majority, that electing majority is guaranteed to share at least one node with any majority that committed an entry — so the new leader is guaranteed to see any already-committed data and never erase it. (Raft's "Leader Completeness" property, which the Raft paper proves, rests on exactly this overlap.)

Analogy: Picture a club with 5 members deciding a rule by show of hands, but votes happen at different times in different rooms and people come and go. You declare a rule "passed" only if *at least 3 of the 5* raised their hand for it. Now suppose a sneaky member later tries to pass a *contradictory* rule, also needing 3 hands. Because $3 + 3 = 6$ is more than the 5 total members, at least one person would have to have raised their hand for *both* contradictory rules — and that person remembers they already committed to the first one and refuses. The majority rule makes it physically impossible for two contradictory rules to both pass. That is exactly why consensus uses majorities.

Common mistake: Using an *even* number of nodes thinking "more nodes = more reliable". A 4-node cluster needs a majority of 3 and tolerates only 1 failure — exactly the same fault tolerance as a 3-node cluster, but with more machines to fail and slower writes. Worse, even sizes make ties more likely. Consensus clusters are almost always **odd** (3, 5, 7) for the best failures-tolerated-per-node ratio.

Common mistake: Believing a network partition can let "both sides keep working". With majority quorums, *at most one side of any partition can contain a majority*. If you split a 5-node cluster into a group of 3 and a group of 2, only the group of 3 can make decisions; the group of 2 cannot reach quorum and must stop accepting writes. This is the algorithm *choosing safety over availability* on the minority side — and it is exactly what prevents split-brain.

Key takeaway: Majority quorums turn the abstract goal "never let two nodes disagree" into a concrete, countable rule. Because any two majorities share a member, and that shared member can't endorse two conflicting values, conflicting decisions are mathematically impossible. This single idea — overlap — is the load-bearing wall under both Raft and Paxos.

Common mistakes & misconceptions

Common mistake: Thinking consensus means "the nodes take a vote and majority wins, once." Real consensus must keep Agreement holding *forever*, across crashes, restarts, partitions, and leader changes — the decision can never be reversed or contradicted later.

Common mistake: Reading FLP impossibility as "consensus can't be done, so Raft/Paxos are broken." FLP forbids only a *guarantee* of termination in a *fully asynchronous, deterministic* model. Raft and Paxos are always safe and terminate in practice; they only forfeit the impossible liveness guarantee, and only during pathological network conditions.

Common mistake: Confusing safety with liveness. "It's slow / stuck" is a liveness problem (recoverable). "Two nodes decided different values" is a safety problem (permanent corruption). Good algorithms sacrifice liveness to protect safety — never the reverse.

Common mistake: Assuming more nodes always means more fault tolerance. A 4-node cluster tolerates the same single failure as a 3-node cluster but is slower and tie-prone. Fault tolerance jumps only at odd sizes ($3 \rightarrow 1$, $5 \rightarrow 2$, $7 \rightarrow 3$).

Common mistake: Thinking a node can tell "crashed" from "slow". In an asynchronous network it fundamentally cannot — that indistinguishability is the entire source of difficulty, and timeouts are only a *guess*, not a fact.

Best practices

Best practice: Run consensus clusters with an *odd* number of nodes (3 or 5 for most workloads). It maximizes failures-tolerated per machine and avoids tie scenarios.

Best practice: When you design or evaluate a consensus-based system, check the two halves separately: "Is it always *safe* (never two conflicting decisions)?" and "Is it *live* during normal operation?" Demand absolute safety; accept that liveness can pause during severe network failures.

Best practice: Do not invent your own consensus protocol. Reuse a proven implementation (etcd/Raft, ZooKeeper/ZAB, Consul) — the subtle overlap and commit rules are exactly where hand-rolled protocols silently violate Agreement.

Best practice: Keep a consensus cluster small and well-connected. Every committed decision needs a round-trip to a majority, so latency and write throughput degrade as you add nodes; scale read capacity with replicas/followers, not by enlarging the voting set.

Best practice: Tune timeouts to your real network's behavior. Too short and healthy-but-slow nodes get falsely suspected (needless leader churn); too long and genuine failures take ages to recover from. The timeout is your practical stand-in for the unsolvable "crashed vs slow" question.

Section summary

- **Consensus** is getting a group of nodes to agree on a single value and never reverse it, despite crashes, lost messages, and delays — it is the foundational hard problem of distributed systems.
- We need it for **leader election, ordering operations, committing transactions, distributed locks, and cluster membership** — anywhere "everyone must agree exactly once" matters.
- A correct consensus protocol must satisfy **Agreement** (no two decide differently), **Validity/Integrity** (the value was actually proposed), and **Termination/Liveness** (a decision is eventually reached).

- **Safety** = "never do a wrong thing" (permanent if violated); **liveness** = "eventually do a good thing" (recoverable). Good algorithms sacrifice liveness, never safety.
- The **FLP impossibility result** proves that in a fully asynchronous, deterministic system with even one possible crash, you cannot *guarantee* consensus always terminates — because you can't distinguish a crashed node from a slow one.
- Real systems escape FLP with **timeouts (partial synchrony), randomization, and failure detectors** — keeping safety absolute and buying back liveness during good network periods.
- Consensus uses **majority quorums** (more than half) because **any two majorities must overlap in at least one node**, and that shared node cannot endorse two conflicting values.
- The overlap property is the mathematical lock that guarantees Agreement and lets new leaders safely inherit committed data — it is the core idea beneath both Raft and Paxos.
- Use **odd-sized, small clusters**; at most one side of any network partition can hold a majority, which is precisely what prevents split-brain.

2. Replicated State Machines & the Log

This section gives you the **one mental model** that every practical consensus system — Raft, Paxos, ZooKeeper's ZAB, etcd, Consul, Google's Chubby and Spanner — is built on. If you understand this section deeply, the later sections on Raft and Paxos become "two different ways to do the same thing." Almost everything those protocols do is in service of the idea you are about to learn.

The idea has a name: the **Replicated State Machine**, usually shortened to **RSM**. Let's build it up slowly.

2.1 What is a "state machine"? (the plain version)

A **state machine** is just a fancy name for a thing that has some *current state* (some data) and a fixed set of *commands* that change that state. You feed it a command, it updates its state, and it may give back a result. That's it.

The crucial property we care about is this: a state machine is **deterministic**. *Deterministic* means "no randomness, no surprises" — given the same starting state and the same command, it always produces the exact same new state and the exact same result. Every single time. No reading the clock, no random numbers, no peeking at the network — just (old state + command) → (new state + result).

Example: Think of a tiny in-memory key-value store (a "KV store") — basically a dictionary. Its *state* is a map like `{ "x": 1, "y": 5 }`. Its *commands* are things like `SET x 10`, `SET y 7`, `DELETE x`, `INCR y`. Start it empty, apply `SET x 1` then `SET x 2` then `INCR x`, and you will *always* end with `{ "x": 3 }`. The result never depends on luck — only on the commands and the order they arrive in.

2.2 The core insight: same start + same commands + same order = same end

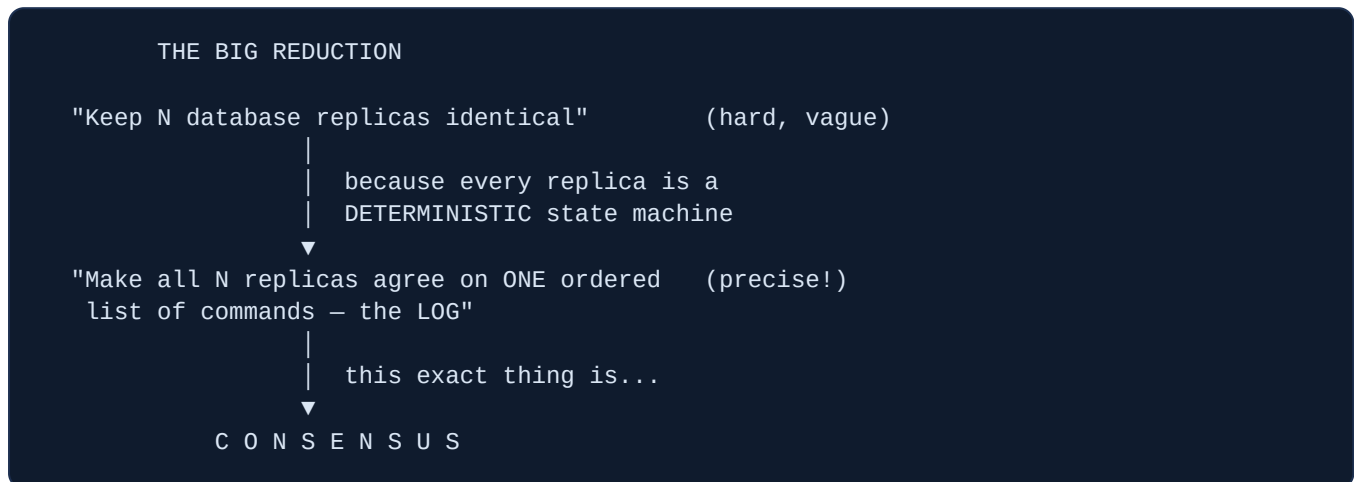
Now copy that KV store onto five different servers. Call each copy a **replica** (you already know this word from the foundations guide — it's an independent server holding its own full copy of the data). We want all five replicas to hold *identical* data, forever, even though servers crash, messages get lost, and there is no global clock.

Here is the whole trick, and it is almost embarrassingly simple:

Key takeaway: If every replica (1) starts in the **same initial state**, and (2) applies the **same commands** in the **same order**, then because each replica is deterministic, they all pass through exactly the same sequence of states and end up **identical**. No replica ever needs to copy another's *data*; they only need to agree on the *list of commands*.

Stare at that for a second, because it changes the whole problem. We started with a hard-sounding goal — "keep five copies of a database in sync across an unreliable network" — and we just reduced it to a different goal: **"make all five servers agree on one single, ordered list of commands."** If they agree on the list, identical data falls out for free.

That ordered list of commands has a name: the **log**. And "make everyone agree on the ordered log" is *exactly* what a consensus algorithm does. So:



Analogy: Imagine five accountants in five different cities, each keeping their own copy of the same ledger by hand. You do *not* fax them photos of your ledger pages to keep them in sync — that's slow and error-prone. Instead, every accountant works from the **same numbered list of instructions**: "#1: deposit \$100. #2: withdraw \$30. #3: transfer \$20...". As long as they all follow the same numbered instructions in the same order, and each does the arithmetic the same way (they're deterministic), their ledgers are guaranteed to match — without anyone ever comparing ledgers. Agreeing on the *numbered instruction list* is the only hard part. That list is the log.

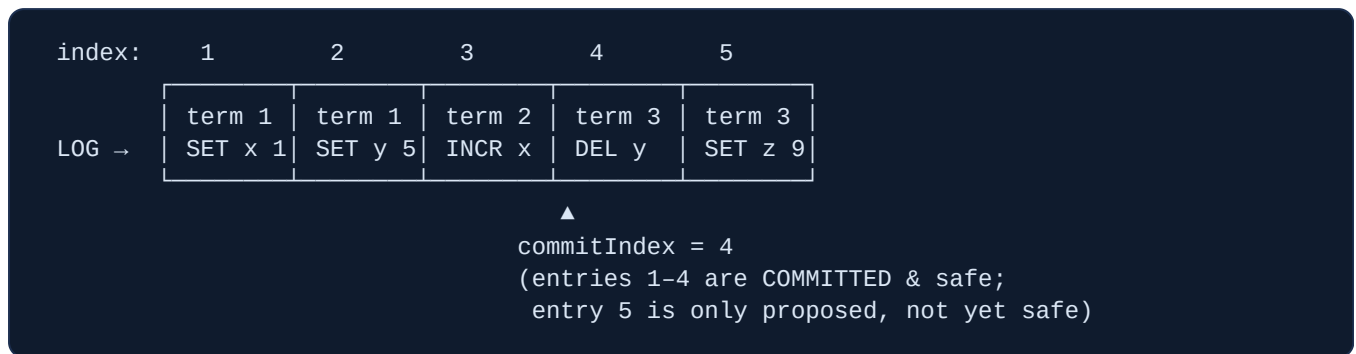
2.3 The replicated log, drawn as boxes

The **log** is an **append-only** list of **entries**. "Append-only" means you only ever add to the end — you never insert in the middle and never edit a committed entry. (Followers can have wrong, uncommitted entries *overwritten*, as we'll see — but agreed-upon history is immutable.)

Each **log entry** carries three things:

- **An index** — the entry's position in the list: 1, 2, 3, 4, ... Think of it as the line number. It defines the *order*.
- **A command** — the actual instruction for the state machine, e.g. `SET x 3`.
- **A term** (Raft's word) / **epoch** (ZooKeeper's word) / **ballot** (Paxos's word) — a number telling you "which leadership period created this entry." We'll explain terms fully in §2.5; for now just know each entry is stamped with one.

Here is one server's log drawn as boxes. Top number = term, bottom = command. The index runs left to right:



2.4 "Committed" vs "applied" — two different milestones

This distinction trips people up constantly, so we'll be precise. An entry passes through milestones on its way to being real:

1. **Appended** — the entry exists in some server's log, but might still be lost or overwritten. Not safe yet.
2. **Committed** — the entry has been safely replicated (stored) on a **majority** of servers. "Majority" means more than half — for 5 servers, at least 3. Once committed, the entry is **permanent**: it can never be lost or changed, even if servers crash, because any future majority must overlap with this majority on at least one server that has the entry. *Committed* = "this decision is final."
3. **Applied** — the command has actually been **run against the local state machine**, changing the data (the KV map). *Applied* = "this decision has taken effect on my copy."

Two pieces of per-server state track these milestones (the real field names from Raft):

- **commitIndex** — the highest index known to be **committed**.
- **lastApplied** — the highest index this server has actually **applied** to its state machine.

The rule every server runs in a loop is simply: **whenever** **commitIndex > lastApplied**, **increment lastApplied** and **apply that next log entry to the state machine**. So "applied" always chases "committed." Commit is the agreement; apply is the side effect.

Common mistake: Thinking "committed" and "applied" are the same thing. They are not. An entry becomes **committed** the instant a majority has it stored — a purely logical fact about replication. It becomes **applied** later, when each server gets around to running it against its data. A follower can know entry 4 is committed (it saw a higher **leaderCommit** from the leader) but not yet have applied it. Commit decides "this is final and ordered"; apply decides "my data now reflects it." Only committed entries may ever be applied — never an uncommitted one, because an uncommitted entry could still be thrown away.

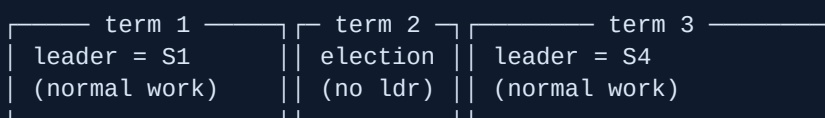
Key takeaway: **Commit** is about *agreement and durability* (a majority stored it → it's final). **Apply** is about *effect* (run it on the state machine → the data changes). Commit comes first and triggers apply. `commitIndex` tracks the first; `lastApplied` tracks the second; `lastApplied` never gets ahead of `commitIndex`.

2.5 Term / epoch / ballot numbers — a logical clock for leadership

Recall from the foundations guide: there is **no global clock**, so we can't order events by wall-clock time. We need a *logical* way to say "this happened in a later leadership period than that." That's what the **term** number is.

A **term** (Raft) — equivalently an **epoch** (ZAB) or **ballot number** (Paxos) — is a **monotonically increasing integer** that labels each period of leadership. "Monotonically increasing" just means it only ever goes *up*, never down: 1, 2, 3, 4, ... Every time the cluster holds a new election to pick a leader, it bumps the number. **A new election = a new term.**

TIME (logical, not wall-clock) →



term goes 1 → 2 → 3 (only ever UP)

Some terms have a leader; some terms are just a failed election (split vote) with no leader — then the cluster moves to the next term.

Why does a single ever-increasing number matter so much? Because it lets every server settle disputes locally with one comparison, and it cleanly evicts stale leaders. Two rules do almost all the work:

- **Higher term always wins.** Every message (every RPC — "Remote Procedure Call," a function call sent over the network to another server) carries its sender's term. When a server receives a message with a term *higher* than its own `currentTerm`, it adopts that higher term and immediately steps down to being a plain follower. When a server receives a message with a term *lower* than its own, it **rejects** it — "you're out of date, ignore me."
- **One vote per term.** A server stores who it voted for this term in a field called `votedFor`, so it can never vote twice in the same term. Combined with "majority needed to win," this guarantees **at most one leader per term** — two candidates can't both collect a majority from the same set of voters in the same term.

This is precisely how the system avoids the nightmare of **two leaders clashing**. Suppose an old leader from term 2 was network-partitioned (cut off) for a while, and meanwhile the rest elected a new leader in term 3. When the partition heals and the old term-2 leader tries to push entries, every healthy server

sees "term 2 < my term 3" and rejects it. The old leader, on hearing a reply carrying term 3, sees the higher number and steps down. No corruption — the higher term simply wins.

Analogy: Term numbers are like the regnal numbers of monarchs — "Henry I, Henry II, Henry III."

Everyone obeys the highest-numbered monarch they've heard of. If a deposed king (Henry II) wanders back into the throne room issuing orders, the guards just say "Sorry, we serve Henry III now" and ignore him. The instant Henry II hears the number "III," he knows he's been superseded and stands down. The number alone — not a clock, not a vote recount — resolves who is legitimately in charge.

Key takeaway: The term/epoch/ballot number is the *logical clock for leadership*. It is monotonic (only goes up), it stamps every entry and every message, "higher term wins / lower term is rejected," and "one vote per term + majority" gives at most one leader per term. This single integer is what makes a stale, returning leader harmless instead of catastrophic.

2.6 Leader-based vs leaderless — why nearly everyone elects a leader

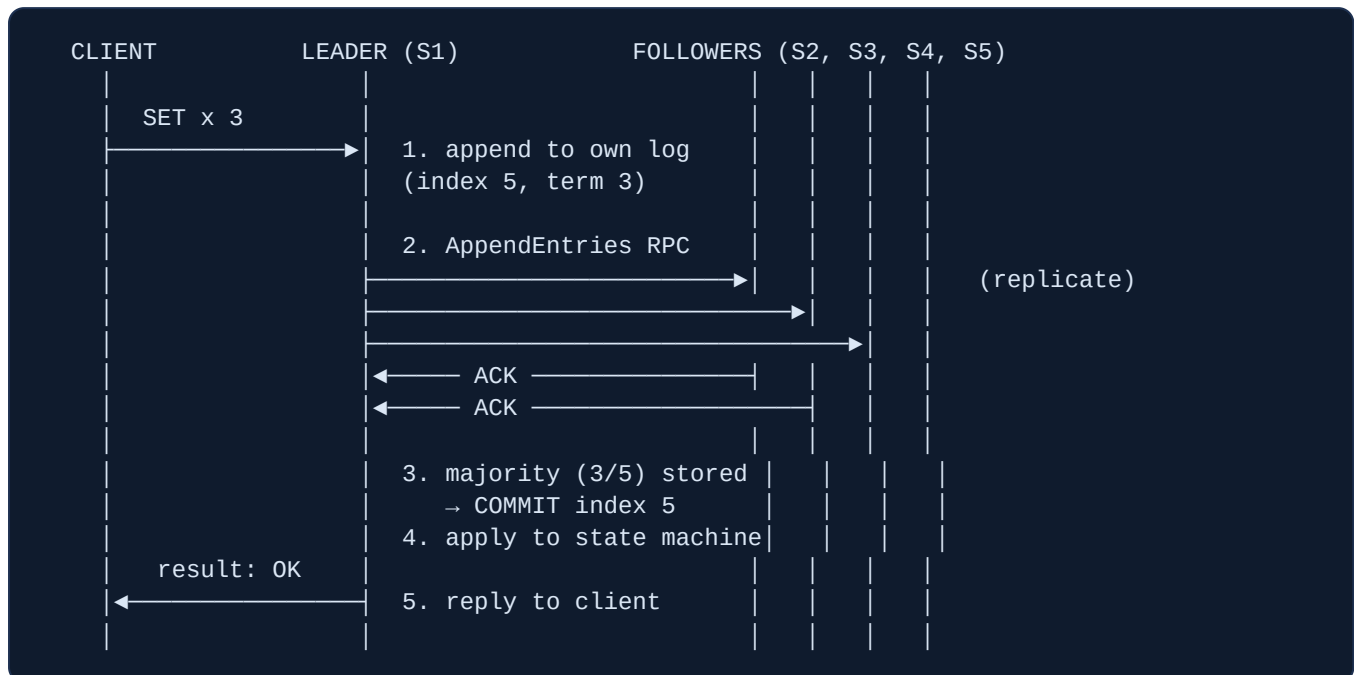
There are two broad ways to agree on the log:

Approach	How it works	Trade-off	Real systems
Leader-based	The cluster elects one server as leader . The leader alone decides the order of entries and pushes them to followers . All writes funnel through the leader.	Simple to reason about; one server sequences everything, so the order is obvious. Downside: the leader is a bottleneck and a single point of (re-elected) failure — if it dies you pause for an election.	Raft, Multi-Paxos (with a stable leader), ZooKeeper/ZAB, etcd, Consul, Chubby
Leaderless	No fixed leader; any server can propose, and servers run a voting round (or several) per individual log slot to agree on what goes there.	No single bottleneck and no election pause. Downside: <i>dueling proposers</i> — two servers proposing for the same slot can keep interrupting each other, causing extra rounds and stalls (livelock).	Basic (single-decree) Paxos; Egalitarian Paxos (EPaxos)

Why do almost all production systems pick a leader? Because once you have a single leader, the order of the log is trivially decided — it's just "whatever order the leader received the requests in." There is no argument about sequencing, because one server is doing all the sequencing. This makes the system far easier to understand, debug, and reason about, and it makes the common case fast: a stable leader can skip the expensive "who's in charge?" negotiation and just stream entries. (This is exactly the optimization **Multi-Paxos** makes over basic Paxos — run the leader-election "prepare" phase once, then reuse that leadership to commit many log entries with a single round-trip each. Raft bakes a strong leader in from the start.)

2.7 How a client actually talks to the system

Now put yourself in the shoes of a **client** — your web application that wants to write data. The normal happy path in a leader-based system is:



1. The client sends its command to the **leader**. (If it accidentally contacts a follower, the follower redirects it to the leader.)
2. The leader **appends** the command to its own log as a new entry (next index, current term).
3. The leader **replicates** the entry to followers via the **AppendEntries** RPC (covered in §2.8).
4. Once a **majority** have stored it, the leader marks it **committed** and **applies** it to its own state machine.
5. The leader **replies** to the client with the result. Followers learn the entry is committed from the leader's next message and apply it too.

Why clients need idempotency / de-duplication

Here's a subtle but critical problem. Suppose the leader commits your **INCR x**, applies it, and then **crashes before the reply reaches you**. Your client times out and **retries**. A new leader is elected — and now your **INCR x** risks being applied *twice*, turning a +1 into a +2. On a replicated state machine, a double-apply corrupts the data on every replica identically, so you can't even detect it by comparing replicas.

The fix is **idempotency via de-duplication**. *Idempotent* means "applying it twice has the same effect as applying it once." The standard technique:

- The client opens a **session** and tags every command with a unique, increasing **serial number** (a request id) — e.g. `{ requestId: 1742, cmd: "INCR x" }`.

- The state machine remembers, per client, the highest serial number it has already applied *and the result it returned*.
- If a command arrives whose serial number was already applied, the system **does not re-run it** — it just returns the remembered result.

This turns "at-least-once delivery" (the network may deliver your retry more than once) into "exactly-once *effect*" (the command changes state only once). This is what people loosely call "exactly-once" — it's really *at-least-once delivery + idempotent processing*.

Analogy: Online payment links use an "idempotency key." If your browser stutters and re-submits the payment form, the bank sees the same key it already processed and replies "already done — here's the original receipt" instead of charging your card twice. The serial number in a consensus client session is exactly that idempotency key, applied to log commands.

Common mistake: Assuming that because consensus gives you a single agreed log, you automatically get exactly-once execution. You don't — not without client de-duplication. Consensus guarantees *the log is the same everywhere*; it does **not** guarantee your retried request appears in that log only once. Two copies of the same retried command are two distinct, validly-committed entries unless you tag requests with ids and have the state machine skip duplicates.

2.8 The two RPCs that move the log (Raft's concrete fields)

To make this real, here are the exact fields of the two messages a leader-based system uses. We use Raft's names because they're the clearest; Paxos and ZAB have equivalents.

AppendEntries — replicate log entries (and act as a heartbeat)

Sent by the leader to followers to copy entries. When sent with an empty `entries[]` list, it doubles as a **heartbeat** ("I'm still alive, don't start an election").

Field	Meaning
<code>term</code>	The leader's current term. Lets a follower reject a stale leader.
<code>leaderId</code>	Who the leader is, so followers can redirect clients to it.
<code>prevLogIndex</code>	The index of the entry <i>immediately before</i> the new ones — the "glue point."
<code>prevLogTerm</code>	The term of the entry at <code>prevLogIndex</code> . Together with <code>prevLogIndex</code> this is the consistency check .
<code>entries[]</code>	The new log entries to store (empty for a heartbeat).
<code>leaderCommit</code>	The leader's <code>commitIndex</code> , so the follower can learn which entries are now committed and may be applied.

RequestVote — ask other servers to elect you leader

Sent by a **candidate** (a server trying to become leader) during an election.

Field	Meaning
<code>term</code>	The candidate's new term (it bumped the number to start this election).
<code>candidateId</code>	Who is asking for the vote.
<code>lastLogIndex</code>	Index of the candidate's last log entry — used to prove its log is "up-to-date."
<code>lastLogTerm</code>	Term of the candidate's last log entry — the primary measure of "up-to-date."

(The leader also keeps two per-follower bookkeeping arrays: `nextIndex[]` — the next index it will try to send each follower — and `matchIndex[]` — the highest index it knows is safely replicated on each follower. These drive the repair process below.)

2.9 The Log Matching Property and repairing a diverging follower

Followers can fall behind or even hold *wrong* entries — for example, a server that briefly followed a leader who crashed before its entries got committed. We need a way to force every follower's log back into agreement with the leader's, safely. Raft guarantees this with the **Log Matching Property**:

Key takeaway (Log Matching Property): If two logs contain an entry at the **same index** with the **same term**, then (a) those two entries store the *identical command*, AND (b) *all entries before them are identical too*. So matching just one (index, term) pair proves the entire history up to that point matches. This is what makes repair efficient: find one matching point and everything before it is already correct.

How the consistency check enforces it: every `AppendEntries` says "before these new entries, you should have an entry at `prevLogIndex` with term `prevLogTerm`." The follower checks: do I actually have that exact entry? If **yes**, the leader's history matches mine up to that point — accept the new entries. If **no**, **reject** the RPC. On rejection the leader **decrements** `nextIndex` for that follower and retries with an earlier `prevLogIndex`, walking backwards one step at a time until it finds the last point where the two logs agree. From there it ships everything forward, and the follower **overwrites** any conflicting entries with the leader's.

Here is the repair, drawn. Server S3 has a diverged tail (a leftover `term 2` entry at index 3 that never committed). The leader (term 3) walks `nextIndex` back to the agreement point and overwrites:

BEFORE REPAIR

	index:	1	2	3	4	5	
LEADER	S1:	[T1 a]	[T1 b]	[T3 c]	[T3 d]	[T3 e]	← source of truth
follower	S2:	[T1 a]	[T1 b]	[T3 c]	[T3 d]	[T3 e]	✓ already matches
follower	S3:	[T1 a]	[T1 b]	[T2 x]			x index 3 term differs
follower	S4:	[T1 a]	[T1 b]				(just behind, no conflict)

Leader sends AppendEntries to S3 with prevLogIndex=4 ... reject (S3 has no #4)
... prevLogIndex=3, prevLogTerm=3 ... reject (S3 has T2 not T3)
... prevLogIndex=2, prevLogTerm=1 ... MATCH! (both [T1 b])
→ agreement point found at index 2. Leader ships indexes 3,4,5.
S3 OVERWRITES its bad [T2 x] at index 3.

AFTER REPAIR

	index:	1	2	3	4	5	
LEADER	S1:	[T1 a]	[T1 b]	[T3 c]	[T3 d]	[T3 e]	
follower	S2:	[T1 a]	[T1 b]	[T3 c]	[T3 d]	[T3 e]	✓
follower	S3:	[T1 a]	[T1 b]	[T3 c]	[T3 d]	[T3 e]	✓ repaired (T2 x discarded)
follower	S4:	[T1 a]	[T1 b]	[T3 c]	[T3 d]	[T3 e]	✓ caught up

└────────── all logs now IDENTICAL ─────────┘
→ apply in order → all state machines IDENTICAL

Example: Walk S3 above. Its bad entry [T2 x] at index 3 came from a previous leader (term 2) that died before that entry reached a majority — so it was never committed, and discarding it loses nothing anyone was promised. The leader's backward walk stops at index 2, where both logs hold [T1 b]. By the Log Matching Property, everything at index 1–2 is therefore already identical, so the leader only needs to resend from index 3. S3 overwrites [T2 x] with [T3 c] and copies [T3 d][T3 e]. Now all five logs are byte-for-byte equal — and because each server applies them in index order to a deterministic state machine, all five KV stores end up holding exactly the same data.

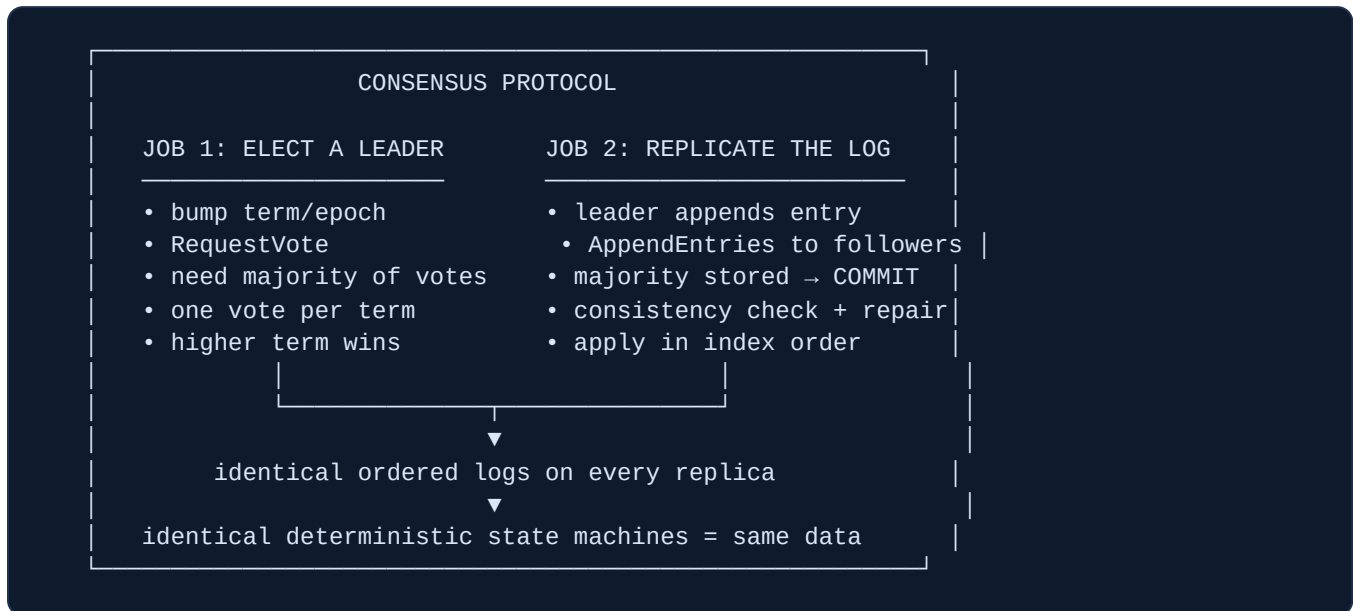
Analogy: Two people are syncing the same novel manuscript. Instead of comparing every page, they flip backwards together until they hit a page that is word-for-word identical in both copies ("ah, page 84 matches"). Everything up to page 84 is therefore already in sync, so they only need to recopy from page 85 onward — and the person who is behind *tears out* their later, conflicting pages and replaces them with the authoritative version. That "flip back to the last matching page, then overwrite forward" is exactly `nextIndex` repair.

2.10 The two jobs of every consensus protocol

Everything above collapses into **two jobs** that every leader-based consensus protocol must do. The rest of this study guide — all of Raft, all of Multi-Paxos, all of ZAB — is just two different ways of doing these same two jobs:

1. **Agree on a leader (leader election).** Pick exactly one server to sequence the log, and make sure stale leaders are harmlessly evicted. The **term/epoch/ballot number** plus **majority voting** are the tools that make "at most one leader per term" true. (This is what `RequestVote` and terms are for.)

2. **Replicate the log safely (log replication).** Get every committed entry onto a majority, in the same index order, and repair any follower that diverges — so that "committed" entries are permanent and identical everywhere. (This is what `AppendEntries`, the consistency check, `commitIndex`, and `nextIndex` repair are for.)



Common mistakes & misconceptions

Common mistake: Believing replicas keep in sync by copying each other's *data*. They don't. They copy the *ordered log of commands* and each replays it independently. Identical data is a *consequence* of identical logs plus deterministic state machines — never something that's directly shipped around.

Common mistake: Letting non-determinism leak into the state machine. If a command does anything that varies between servers — reads the local clock, generates a random id, calls an external API, depends on map-iteration order — the replicas diverge *even with identical logs*. Any such value must be decided **once by the leader and written into the log entry** (e.g. log "set createdAt = 2026-06-21T10:00:00Z", not "set createdAt = now()"), so every replica applies the same fixed value.

Common mistake: Confusing "committed" with "applied." Committed = a majority stored it, so it is final and ordered. Applied = a server has run it against its data. Commit happens first and is the agreement; apply is the later side effect. Only committed entries may be applied, and `lastApplied` never overtakes `commitIndex`.

Common mistake: Thinking a returning old leader can corrupt data. It can't, thanks to terms. Its messages carry a *lower* term, so healthy servers reject them, and the moment it hears a higher term in any reply it steps down. The monotonic term number — not timing or luck — is what makes stale leaders harmless.

Common mistake: Assuming consensus alone gives exactly-once execution. Consensus makes the *log* identical everywhere; it does not stop a retried client request from being committed twice. You need client sessions with request ids (idempotency keys) and a state machine that skips already-applied serial numbers.

Common mistake: Treating the whole log as immutable. *Committed* history is immutable, yes — but a follower's *uncommitted* tail can be legitimately overwritten by the leader during repair. Those discarded entries were never committed, so no client was ever told they succeeded, and nothing is lost.

Best practices

Best practice: Keep your state machine strictly deterministic. Forbid clock reads, randomness, and external calls inside command application. Resolve every such value at the leader and bake it into the log entry, so all replicas apply identical inputs.

Best practice: Always give clients sessions with monotonic request ids, and have the state machine record "highest serial applied + result returned" per client. This is the cheap, standard way to turn unavoidable retries into exactly-once effects and avoid silent double-applies.

Best practice: Use an odd cluster size (3, 5, 7). Majority for 3 is 2, for 5 is 3 — an odd count maximizes the failures you tolerate per node you pay for (5 nodes tolerate 2 failures; 6 nodes also only tolerate 2). It also avoids tie votes.

Best practice: Never let unbounded logs grow forever. Pair the log with **snapshots** (a compact dump of the state machine at some index) so old, already-applied entries can be discarded and a lagging or new follower can be caught up by snapshot + tail instead of replaying from index 1.

Best practice: Route all writes through the leader and accept the redirect cost. Resist "let any node accept writes" shortcuts — single-leader sequencing is what makes the log order unambiguous and the system easy to reason about, which is why etcd, ZooKeeper, Consul, and Chubby all do it.

Section summary

- A **Replicated State Machine (RSM)**: identical deterministic state machines that start equal and apply the **same commands in the same order** stay equal forever — so consensus reduces to "agree on the ordered log."
- The **log** is an append-only list of **entries**; each entry has an **index** (order), a **command**, and a **term/epoch/ballot** (which leadership period created it).

- **Committed** (a majority has stored it → final and permanent) is distinct from **applied** (run against the local state machine); `commitIndex` tracks the first, `lastApplied` the second, and apply only ever follows commit.
- **Term/epoch/ballot** is a monotonic logical clock for leadership: "a new election = a new term," higher term wins, lower term is rejected, one vote per term → at most one leader per term, and stale leaders are evicted harmlessly.
- Almost all production systems are **leader-based** (Raft, Multi-Paxos, ZAB/ZooKeeper, etcd, Consul, Chubby) because one leader makes log order trivial and the common case fast; leaderless designs avoid the bottleneck but risk dueling proposers.
- Clients send commands to the **leader**, which appends → replicates (`AppendEntries`) → commits on majority → applies → replies; followers learn the commit point from `LeaderCommit` .
- **Idempotency via request ids / client sessions** is required to stop retried commands from being applied twice — consensus alone does not give exactly-once execution.
- The **Log Matching Property** (same index + same term ⇒ identical entry and identical history before it) plus the `prevLogIndex` / `prevLogTerm` consistency check let a leader repair a diverging follower by walking `nextIndex` back to the agreement point and overwriting its uncommitted tail.
- Every consensus protocol has exactly **two jobs**: (1) **agree on a leader** and (2) **replicate the log safely** — Raft and Paxos are just two ways of doing these same two things.

3. Raft — Leader Election

Raft is an algorithm for keeping a **replicated log** in agreement across a cluster of servers. A "replicated log" is just an ordered list of commands (like "set x = 3", then "set y = 9") that every server must store in the same order, so that every server ends up in the same state. The hard part is agreeing on that order when machines crash, restart, and the network drops or delays messages. That agreement problem is called **consensus**.

Raft was created by Diego Ongaro and John Ousterhout (Stanford, 2014) with one explicit, unusual goal: **understandability**. Earlier algorithms (Paxos) are famously hard to reason about, so Raft deliberately breaks the problem into three smaller, mostly independent pieces:

1. **Leader election** — pick exactly one server to be in charge. (This section.)
2. **Log replication** — the leader copies its log to everyone. (Later section.)
3. **Safety** — guarantee the leader can never overwrite already-agreed history. (Later section.)

Raft's central design choice is the **strong leader**: at any moment there is one leader, and *all* changes to the log flow only from the leader outward to the others. Followers never talk to each other about log content. This single rule removes a huge amount of complexity — but it means everything depends on reliably electing a leader, and re-electing one the instant the old leader dies. That is what this section is about.

Analogy: Think of a group writing a shared document by phone. If everyone edits and dictates at once, chaos. So the group agrees: one person is the "scribe" at a time. The scribe is the only one who writes; everyone else just listens and copies. If the scribe goes silent, the group must quickly, and without a referee in the room, pick a new scribe — and crucially, never end up with two scribes writing different things at the same time. Raft is the precise protocol for choosing that scribe.

3.1 The three server states

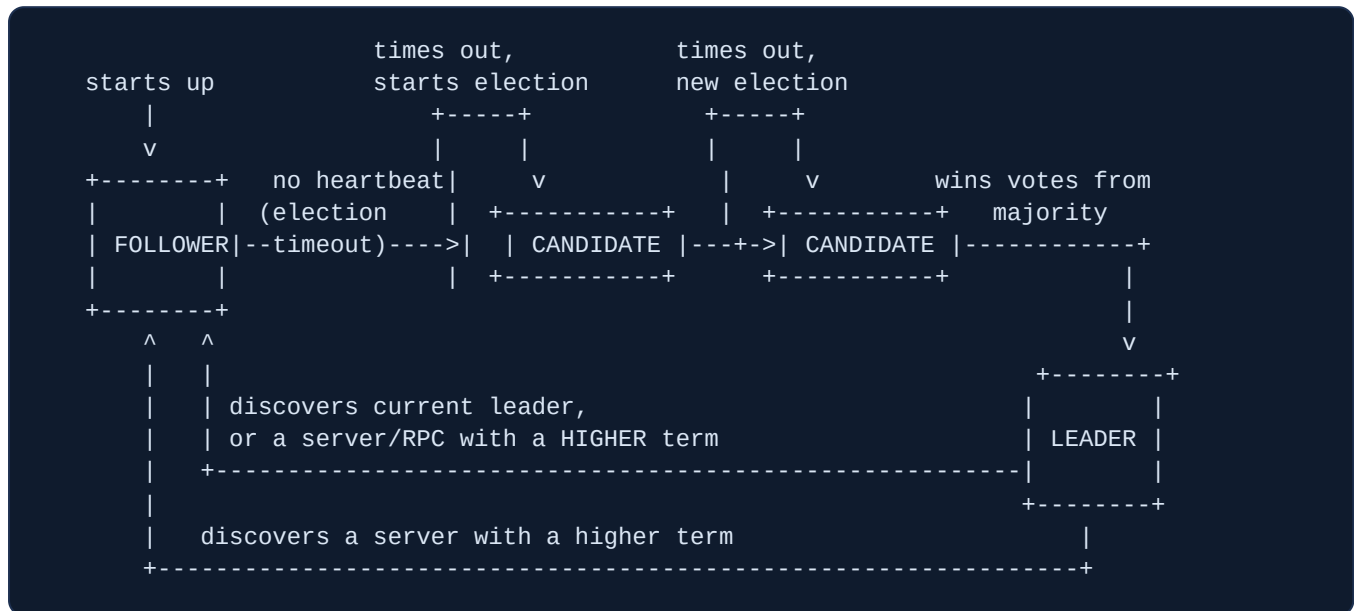
At every moment, each Raft server is in exactly one of three **states** (a "state" here just means "the role this server is currently playing"):

State	What it does	How many at once
Follower	Passive. Issues no requests on its own. Only <i>responds</i> to messages from leaders and candidates. Redirects clients to the leader.	Usually all-but-one
Candidate	A follower that has decided to try to become leader. It is asking the others to vote for it.	Zero or more (briefly)
Leader	Handles all client requests, owns the log, and continuously tells followers it is alive.	At most one per term

In normal, healthy operation there is **exactly one leader and every other server is a follower**. The candidate state exists only during the brief, occasional moment of electing a new leader.

The state machine (transitions)

A server moves between these states based on what it hears (or fails to hear) from the network. Here is the full state diagram — this is the heart of leader election:



Read the transitions as plain rules:

- **Follower** → **Candidate**: a follower hears nothing from any leader for too long (its *election timeout* fires), so it starts an election.
- **Candidate** → **Leader**: the candidate collects votes from a *majority* of the cluster.
- **Candidate** → **Follower**: the candidate discovers a legitimate current leader (or any message carrying a higher term — defined below), so it gives up and steps down.
- **Candidate** → **Candidate**: the election timed out with no winner (a *split vote*), so the candidate starts a fresh election.
- **Leader** → **Follower**: the leader learns of a higher term than its own, meaning a newer election has happened, so it steps down immediately.

Key takeaway: Every server is a Follower, Candidate, or Leader. Followers are passive; they only become Candidates when they stop hearing from a leader. There is no central coordinator deciding who's leader — the servers self-organize entirely through these transitions.

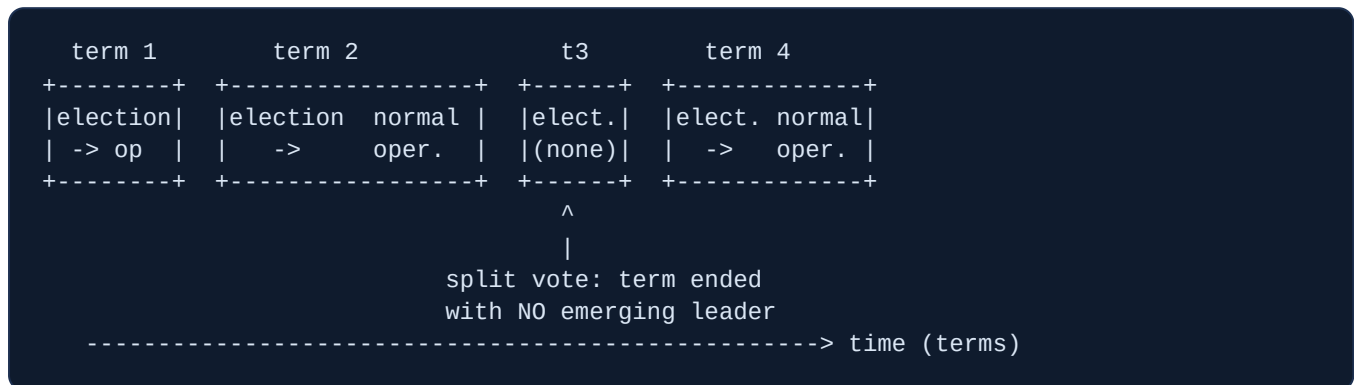
3.2 Terms — Raft's logical clock

Before we can run an election, we need the concept of a **term**.

What it is: a term is a numbered period of time. Terms are numbered with consecutive integers: term 1, term 2, term 3, and so on. Each term **begins with an election**. If the election produces a winner, that

server is leader for the rest of that term. If the election fails to produce a winner, the term ends with no leader and the next term (with a new election) begins.

The problem it solves: there is no global clock in a distributed system (you already know this). Servers can't agree on wall-clock time. But they *can* agree on "which election are we on?" The term number is a **logical clock** — a counter that only moves forward — that lets every server detect stale information. A message from an old term is obviously out of date.



Notice term 3 (t3) is short and has **no leader** — that's an election that failed (a split vote). Raft handles this gracefully: it just rolls into term 4 and tries again.

Two rules make terms powerful:

- **At most one leader per term** (the *Election Safety* property). A term might have zero leaders (failed election) but never two. We'll prove why below.
- **Every message carries the sender's term number.** Every server stores its own `currentTerm`.

When two servers communicate, they compare terms:

- If a server sees a term *larger* than its own `currentTerm`, it updates `currentTerm` to the larger value and, if it was a candidate or leader, immediately reverts to **follower**. (It has been left behind by a newer election.)
- If a server receives a request with a *stale* (smaller) term, it **rejects** the request.

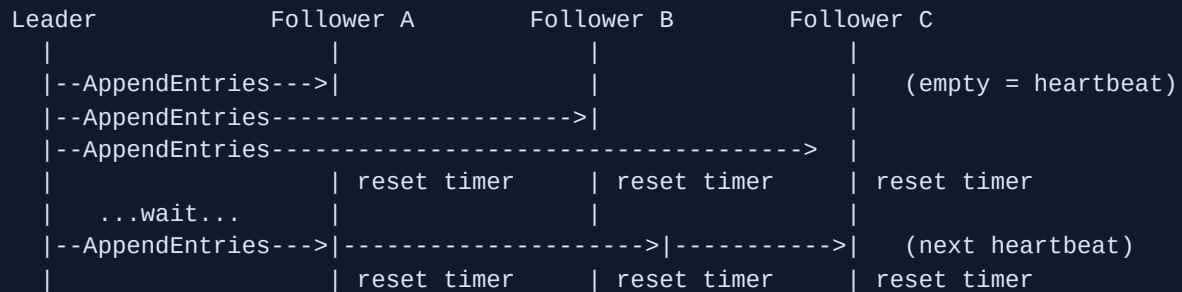
Analogy: A term is like a "presidential term" with a number. Every official letter is stamped with the term number it was written in. If you receive a letter stamped "Term 4" but you thought it was still Term 2, you immediately realize you missed an election — you accept that Term 4 is current and stand down from any office you held. If someone hands you a letter stamped "Term 2" today, you ignore it as obsolete.

3.3 Heartbeats — how the leader stays in power

How do followers know the leader is still alive? Through **heartbeats**.

The leader periodically sends an `AppendEntries` RPC to every follower. (An **RPC**, "remote procedure call", is just a message that asks another server to run something and reply.) `AppendEntries` is the same RPC the leader uses to replicate log entries — but when it carries **no log entries** (an empty `entries[]` list), it functions purely as a **heartbeat**: a "I'm still here, stay calm" signal.

Each time a follower receives a valid `AppendEntries` from the current leader, it **resets its election timer**. As long as heartbeats keep arriving, no follower ever times out, so no follower ever starts an election, so the leader keeps its job. The leader sends heartbeats even when idle (no client requests), precisely to prevent followers from timing out.



Key takeaway: A Raft leader holds power by *continuously broadcasting that it is alive*. Silence — not an explicit "I'm dying" message — is what triggers a new election. This means Raft tolerates a leader that crashes, freezes, or gets network-partitioned away: in all three cases the followers simply stop hearing heartbeats and react identically.

3.4 The election timeout — what kicks off an election

The **election timeout** is the amount of time a follower waits, hearing nothing from a leader, before it gives up and tries to become leader itself.

If a follower's election timeout elapses without receiving any `AppendEntries` from a current leader (and without granting a vote to a candidate — more on that soon), it assumes there is no viable leader and begins an election. To begin an election the follower does four things, in order:

1. **Transitions to candidate** state.
2. **Increments its `currentTerm`** (e.g. from 2 to 3) — starting a brand-new term.
3. **Votes for itself** (sets its `votedFor` to its own id, and counts one vote).
4. **Resets its election timer** and sends `RequestVote` RPCs, in parallel, to every other server.

The RequestVote RPC — exact fields

`RequestVote` is the RPC a candidate sends to ask "please vote for me as leader." Here are its exact arguments and results, straight from the Raft specification:

Direction	Field	Meaning (plain words)
Argument	<code>term</code>	The candidate's term number (the new, incremented one).
Argument	<code>candidateId</code>	Which server is requesting the vote (so the voter can record it).
Argument	<code>lastLogIndex</code>	The position (index) of the last entry in the candidate's log — how <i>long</i> its log is.
Argument	<code>lastLogTerm</code>	The term of that last log entry — how <i>recent</i> its log is.
Result	<code>term</code>	The voter's <code>currentTerm</code> , so a behind candidate can update itself.
Result	<code>voteGranted</code>	<code>true</code> means "yes, you have my vote."

The last two arguments — `lastLogIndex` and `lastLogTerm` — exist for the safety restriction we preview in §3.7. For now, just note that the candidate advertises how up-to-date its log is.

3.5 Winning the election — majority and at-most-one-vote

A candidate **wins** the election if it receives `voteGranted = true` from a **majority** of *all* servers in the cluster (not just a majority of those that replied). "Majority" means more than half. In a 5-server cluster, a majority is 3.

The voting rule on each server is simple and strict:

- Each server may vote for **at most one candidate per term**, on a **first-come-first-served** basis. It records its choice in `votedFor`. Once it has voted in term *T*, it will refuse any other candidate in term *T*.
- A server rejects a `RequestVote` whose `term` is smaller than its own `currentTerm` (stale candidate).
- (Plus the up-to-date-log check from §3.7.)

When a candidate gathers a majority, it immediately becomes **leader** and starts sending heartbeats to assert its authority and stop any other elections from starting.

Why "at most one leader per term" is guaranteed

This is the elegant part. Suppose, for contradiction, two different candidates both won the *same* term. Each would need votes from a majority of the cluster. But any two majorities of the same set **must overlap in at least one server** (two groups each bigger than half can't be disjoint). That overlapping server would have had to vote *twice* in the same term — which the "at most one vote per term" rule forbids. Contradiction. Therefore at most one candidate can win a given term. This is the **Election Safety** property.

```

5 servers:   S1  S2  S3  S4  S5

Candidate A's majority: [ S1  S2  S3 ]
Candidate B's majority:      [ S3  S4  S5 ]
                        ^ ^
                        |
S3 is in BOTH — it cannot have
voted for both A and B in one term.
=> one of them cannot reach a majority.

```

Analogy: Imagine 5 club members electing a president. Each member gets one ballot per election round and can spend it on only one candidate. To win you need 3 ballots. Two candidates can't *both* collect 3 of the 5 ballots, because $3 + 3 = 6$ ballots but only 5 exist — at least one member would have had to vote twice, which isn't allowed. So at most one winner per round. That's exactly the quorum-overlap argument you already know, applied to votes.

A full election timeline

```

S1(Leader, term2)  S2(Follower)  S3(Follower)  S4(Follower)  S5(Follower)
|                  |              |              |              |
|--heartbeat----->|----->|----->|----->|
X <-- S1 CRASHES   |              |              |              |
                    | ..no heartbeats arriving... |              |
                    | timer fires (S2 first)         |              |
                    | term 2 -> 3, vote self, become CANDIDATE |              |
                    |--RequestVote(term=3, id=S2)----->| (to all)
                    |              |              |              |
                    |<--voteGranted--|<--granted----|<--granted----|
                    | got votes: S2(self)+S3+S4 = 3 of 5 = MAJORITY
                    | become LEADER of term 3
                    |--heartbeat(term=3)----->| ..----->|
                    | everyone resets timers, recognizes S2

```

Sequence: leader dies → a follower times out → it becomes a candidate in a new term → it sends `RequestVote` to all → it collects a majority → it becomes the new leader → its heartbeats restore calm. Total time is typically a few hundred milliseconds.

3.6 Split votes and randomized timeouts

There is a third possible outcome of an election: the candidate **neither wins nor loses**. This is a **split vote**.

How it happens: if several followers time out at nearly the same instant, they all become candidates in the same new term and all start begging for votes simultaneously. The votes get divided among them, and no single candidate reaches a majority. Each candidate then sits, having voted for itself, unable to win and unwilling to vote for anyone else this term.

5 servers. S2 and S4 time out at the SAME time -> both candidates, term 3.

```
S1          S2(cand)      S3          S4(cand)      S5
|    <--ReqVote-- |      |      --ReqVote-->|      |
|              |  --ReqVote--> |              |  <--ReqVote--|... (crossing)

Votes land:  S2 gets: S2(self) + S1    = 2
              S4 gets: S4(self) + S5    = 2
              S3 already voted for S2? or S4? say it voted S4 -> S4=3?
(In the classic split:) S2 = S1,S2 | S4 = S4,S5 | S3 hasn't decided
-> 2 vs 2, nobody reaches 3. *** SPLIT VOTE ***
-> term 3 ends with NO leader. Everyone waits, times out again.
```

Without a countermeasure, split votes could repeat forever: every candidate times out at the same time again, splits again, term after term. Raft's fix is beautifully simple — **randomized election timeouts**.

Instead of every server using the same fixed timeout, each server picks its election timeout **randomly from a fixed range** — the paper uses **150–300 ms** as an example. So the timeouts are spread out. In most cases, *one* server times out clearly before any other, wins the whole election uncontested, and sends heartbeats before anyone else even wakes up. The same randomization is reused after a split: each candidate restarts its timer with a fresh random value before the next attempt, so the next round is very unlikely to tie again.

Split happened in term 3. Now each picks a NEW random timeout for term 4:

```
S2 random timeout = 270ms |-----|fires
S4 random timeout = 165ms |-----|fires <-- WINS THE RACE
                        |
                        S4 times out first, becomes candidate (term 4),
                        collects a majority, and sends heartbeats
                        BEFORE S2's 270ms is up -> S2 hears S4's heartbeat,
                        resets its timer, stays a follower. No second split.
                        -----> time
```

This is a **probabilistic** guarantee, not a hard one: a tie is *possible* in any single round, but its probability shrinks rapidly because two independent random picks rarely match closely, and each retry re-rolls the dice. In practice elections resolve in one or two rounds.

Analogy: Two people reaching for the last slice of pizza at the exact same instant freeze, both say "you take it — no, you," and stall (the split). The social fix: each silently counts to a random number before grabbing again. Whoever finishes counting first just takes it, cleanly, while the other is still counting — and seeing it gone, gives up. Randomized waiting breaks the symmetry that caused the deadlock.

Common mistake: Thinking randomized timeouts *prevent* split votes. They don't prevent them — they make repeated splits exponentially unlikely and make each one resolve fast. Raft tolerates the occasional split (the term just ends leaderless and a new one starts); it only needs splits to be *rare and short*, not impossible.

3.7 The "up-to-date log" voting restriction (preview)

So far a follower grants its vote to the first candidate that asks in a term. But there is one more condition, and it's the bridge to Raft's safety guarantees. A server will grant its vote **only if the candidate's log is at least as up-to-date as its own log**.

The problem it solves: the leader is the single source of truth for the log, and Raft never lets a leader overwrite committed history. So we must never elect a leader that is *missing* entries other servers have already committed — otherwise the new leader's incomplete log would become the official one, and agreed-upon commands would vanish. The vote is the gate that keeps a behind server from ever becoming leader.

How "up-to-date" is compared (this is exactly why `lastLogTerm` and `lastLogIndex` are in `RequestVote`):

1. Compare the **term of the last entry** first. The log whose last entry has the *higher* `lastLogTerm` is more up-to-date.
2. If the last terms are **equal**, then the **longer** log (higher `lastLogIndex`) is more up-to-date.

A voter grants its vote only when the candidate's `(lastLogTerm, lastLogIndex)` is greater than or equal to the voter's own. If the candidate is behind on either measure, the voter says `voteGranted = false`.

Example: Voter S3's last log entry is *index 5, term 4*. Two candidates ask for its vote:

- **Candidate A:** last entry index 6, term 4. Same last term (4), longer log ($6 \geq 5$) → at least as up-to-date → S3 **grants** the vote.
- **Candidate B:** last entry index 8, term 3. Longer log (8), but its last term (3) is *lower* than S3's (4). Term is compared first, so B is **behind** despite being longer → S3 **refuses**. B has a longer-but-older log and must not lead.

This shows why term beats length: a long log full of entries from an old term can still be missing newer, already-committed entries.

We only preview this here. *Why* this exact comparison is sufficient to guarantee that a newly elected leader holds every committed entry — the Leader Completeness property — is the subject of the safety section. For now, hold onto this: **elections aren't just popularity contests; a candidate must also prove its log is current before anyone will vote for it.**

Key takeaway: Leader election and log safety are linked through one rule — the up-to-date-log check in `RequestVote`. Majority voting decides *who* leads; the log comparison decides *who is even eligible* to lead. Together they ensure the winner never destroys committed data.

Common mistakes & misconceptions

Common mistake: Believing a candidate needs a majority of the servers that *respond*. It needs votes from a majority of the **entire cluster** (size is fixed, e.g. 3 of 5). Unreachable servers count as non-votes, not as a smaller denominator. This is what lets a partitioned minority correctly fail to elect a leader.

Common mistake: Thinking a leader announces its own death so the cluster can react. It does not. Detection is by **absence of heartbeats**. A crashed leader, a frozen leader, and a network-isolated leader all look identical to followers — which is exactly why the same timeout mechanism handles all of them.

Common mistake: Assuming every term has a leader. A term can end with **no leader at all** if its election split. Raft simply moves to the next term and tries again; a leaderless term is normal and harmless.

Common mistake: Treating randomized timeouts as a guarantee against ties. They are a *probabilistic* tool. Two candidates *can* still split a vote in one round; randomization just makes that rare and self-correcting, not impossible.

Common mistake: Thinking the most "popular" or first-asking candidate always wins. A candidate with a stale or behind log will be **refused** by up-to-date voters because of the `lastLogTerm / lastLogIndex` check, even if it asked first. Eligibility comes before popularity.

Best practices

Best practice: Set the election timeout range well above the typical round-trip time and heartbeat interval — the rule of thumb is `broadcastTime << electionTimeout << MTBF` (mean time between failures). The paper's 150–300 ms suits a fast LAN; use larger values for cross-datacenter links so transient slowness doesn't trigger needless elections.

Best practice: Send heartbeats far more frequently than the election timeout — commonly an order of magnitude faster (e.g. heartbeat every ~50 ms against a 150–300 ms timeout) — so one dropped heartbeat never costs you a leader.

Best practice: Keep the timeout interval genuinely *randomized per server and per attempt*. Re-roll the random value at the start of every election. A fixed or poorly seeded timeout reintroduces the synchronized split votes randomization was meant to eliminate.

Best practice: Persist `currentTerm` and `votedFor` to stable storage *before* responding to any RPC. If a server crashes and restarts within a term, it must remember it already voted, or it could vote a second time and break the at-most-one-leader guarantee.

Best practice: Prefer an odd cluster size (3, 5, 7). Odd sizes give a clean majority and maximize fault tolerance per node — 5 servers tolerate 2 failures, while going to 6 still only tolerates 2 but makes every quorum larger and slower.

Section summary

- Every Raft server is a **Follower**, **Candidate**, or **Leader**; normal operation has exactly one leader and the rest followers.
- **Terms** are an ever-increasing logical clock; each term starts with an election and has **at most one leader**. Every message carries a term, and a higher term forces stale leaders/candidates back to follower.
- The leader keeps power by broadcasting **heartbeats** (empty `AppendEntries`); receiving one resets a follower's election timer.
- A follower whose **election timeout** elapses becomes a candidate: it increments its term, votes for itself, and sends `RequestVote(term, candidateId, lastLogIndex, lastLogTerm)` to all servers.
- A candidate wins by collecting votes from a **majority of the whole cluster**; each server votes **once per term, first-come**, which (via quorum overlap) guarantees at most one winner.
- **Split votes** happen when servers time out together; **randomized timeouts (e.g. 150–300 ms)** de-synchronize them so one candidate usually wins quickly and repeated splits are rare.
- An election can end with no leader; Raft just starts a new term and retries.
- The **up-to-date-log restriction** — comparing `lastLogTerm` then `lastLogIndex` — means a candidate whose log is behind cannot win, linking election to safety and setting up the Leader Completeness guarantee.

4. Raft — Log Replication, Safety & Membership

In the previous section you saw how Raft picks a single **leader** (the one server allowed to accept new commands) and how a **term** (a numbered period of time, each with at most one leader) bounds the chaos. This section covers the harder half of Raft: once you have a leader, how do you copy the stream of commands to every server so that all of them end up identical — and stay identical even when servers crash, fall behind, or you add and remove machines? That stream of commands is called the **replicated log**, and keeping it consistent is the whole job.

Let me define the core vocabulary up front, in plain words, so the rest reads smoothly.

- **Log** — an ordered list of **entries**. Each server keeps its own copy. The goal is to make every copy identical.
- **Log entry** — one slot in the log. It holds three things: a **command** (the actual operation to run, e.g. "set x = 5"), the **term** in which the leader created it, and its position number, the **index** (1, 2, 3, ...).
- **State machine** — the thing the log feeds. It is just your application's data plus the rule "apply commands in log order." If every server applies the same commands in the same order, every server reaches the same final state. That is the entire point of consensus.
- **Apply / commit** — an entry is **committed** once Raft guarantees it will never be lost; only then is it safe to feed ("apply") it to the state machine and reply to the client.
- **commitIndex** — the highest log index a server knows is committed.
- **Majority** — more than half the servers. In a 5-server cluster a majority is 3. Any two majorities always share at least one server — that overlap is the mathematical trick everything below rests on.

A replicated log (5-server cluster). Each box = one entry: term^index / command

	idx:	1	2	3	4	5	
Leader		[1 x=3]	[1 y=1]	[2 x=9]	[3 z=7]	[3 x=0]	
Follower A		[1 x=3]	[1 y=1]	[2 x=9]	[3 z=7]	[3 x=0]	(in sync)
Follower B		[1 x=3]	[1 y=1]	[2 x=9]			(lagging by 2)
Follower C		[1 x=3]	[1 y=1]	[2 x=9]	[3 z=7]	[3 x=0]	
Follower D		[1 x=3]	[1 y=1]				(lagging by 3)

"1|x=3" means: created in TERM 1, command is x=3, sitting at INDEX 1.
The number before the bar is the TERM, not the value.

Analogy: Think of the log as a shared notebook that a team is copying by hand. The leader is the one person allowed to write a new line. Everyone else copies the leader's notebook line for line. The trouble is that copiers crash, lose pages, or copy stale versions — so the leader needs a disciplined protocol to detect "your page 4 doesn't match mine" and fix it without ever tearing out a line that the team has already officially agreed on.

4.1 The AppendEntries RPC in detail

An **RPC** ("remote procedure call") is just a message one server sends another that says "run this and tell me the result." Raft uses only two RPCs. You met `RequestVote` in the election section. The workhorse here is `AppendEntries`. The leader sends it to every follower both to *copy new entries* and, when there are no new entries, as a **heartbeat** (an empty `AppendEntries` that says "I'm still leader, stay quiet").

Here are the fields the leader puts in each `AppendEntries` request:

Field	Plain meaning
<code>term</code>	The leader's current term. Lets the follower detect a stale (old) leader.
<code>leaderId</code>	Who the leader is, so a follower can redirect clients to it.
<code>prevLogIndex</code>	The index of the entry <i>immediately before</i> the new ones. This is the "join point."
<code>prevLogTerm</code>	The term of that entry at <code>prevLogIndex</code> . Together with <code>prevLogIndex</code> it is a fingerprint of the follower's log up to that point.
<code>entries[]</code>	The new entries to store (empty for a heartbeat).
<code>leaderCommit</code>	The leader's <code>commitIndex</code> — tells the follower how far it is safe to apply.

The reply is tiny: the follower's own `term` (so the leader can step down if it's behind) and a boolean `success`.

When a follower receives `AppendEntries`, it runs these steps in order:

1. **Term check.** If the request's `term` is older than the follower's own term, reply `success = false` — this is a stale leader; ignore it. Otherwise accept this leader and reset the election timeout (so it won't start its own election).
2. **Consistency check.** Look at the follower's own entry at `prevLogIndex`. If the follower has no entry there, or it has one but its term $\neq$ `prevLogTerm`, reply `success = false`. The logs disagree at the join point, so appending now would create a hole or a mismatch.
3. **Repair conflicts.** If an existing entry conflicts with a new one (same index, different term), delete that entry and everything after it. The leader's copy always wins.
4. **Append.** Add any new entries not already present.
5. **Advance commit.** If `leaderCommit` > the follower's `commitIndex`, set `commitIndex = min(leaderCommit, index of last new entry)`, then apply newly-committed entries to the state machine.
6. Reply `success = true`.

The Log Matching Property — why the consistency check is enough

Step 2 looks almost too simple — it only checks *one* entry, the one at `prevLogIndex`. How can checking a single entry guarantee the *whole* log up to that point matches? Because of an invariant Raft

maintains called the **Log Matching Property**:

If two logs contain an entry with the same index and the same term, then (a) they store the same command at that index, and (b) the logs are identical in every entry before it.

Part (a) holds because a leader creates at most one entry per index per term, and entries never move once created. Part (b) holds by induction, and the `AppendEntries` consistency check is exactly what enforces it: a follower only accepts new entries if its entry at `prevLogIndex` already matches the leader's term there. So matching at the join point implies matching all the way back. This is the load-bearing property of the whole protocol — it turns "are these two long logs identical?" into a single cheap comparison.

Key takeaway: `prevLogIndex` + `prevLogTerm` act as a checksum of everything before the new entries. If they match, the entire prefix matches — that is the Log Matching Property, and it lets Raft verify and repair logs one entry at a time instead of comparing the whole thing.

4.2 Consistency check & repair — fixing a divergent follower

After a crash, a follower's log can be wrong in two ways: it can be **missing** entries the leader has, or it can have **extra** entries (from a previous leader) that were never committed and must be thrown away. Raft fixes both with one mechanism.

The leader keeps, for each follower, a number called `nextIndex`: "the next log index I will try to send you." When a leader is first elected, it optimistically sets every follower's `nextIndex` to its own last index + 1. It then sends `AppendEntries` with `prevLogIndex = nextIndex - 1`. If the follower rejects (the consistency check fails), the leader **decrements** `nextIndex` and retries. It keeps backing up until it finds the last index where the two logs agree. From that point forward, its entries flow in, and the follower's step 3 deletes any conflicting tail. Eventually the follower's log is byte-for-byte the leader's.

LOG REPAIR: leader backs up `nextIndex` until logs agree, then overwrites the tail.

```
      idx: 1    2    3    4    5    6    7
Leader (term 4) [ 1 ][ 1 ][ 1 ][ 4 ][ 4 ][ 4 ][ 4 ]  (numbers = TERM of entry)
Follower X      [ 1 ][ 1 ][ 1 ][ 2 ][ 2 ][ 3 ]
                  ^^^^^^^^^^^^^^^^^
                  extra, UNCOMMITTED entries from old terms 2 & 3
```

```
Leader starts: nextIndex(X) = 7 -> AppendEntries prevLogIndex=6,prevLogTerm=4
Follower X@6 has term 3, not 4 -> success=false. Leader: nextIndex 7->6
prevLogIndex=5,prevLogTerm=4 : X@5 has term 2 -> false. nextIndex 6->5
prevLogIndex=4,prevLogTerm=4 : X@4 has term 2 -> false. nextIndex 5->4
prevLogIndex=3,prevLogTerm=1 : X@3 has term 1 -> MATCH! success=true.
Leader now sends entries[4..7] = (4,4,4,4).
Follower deletes its old 4,5,6 (terms 2,2,3) and writes the leader's.
```

```
Result -> Follower X [ 1 ][ 1 ][ 1 ][ 4 ][ 4 ][ 4 ][ 4 ]  identical.
```

Example (missing entries, the easy case): Follower D has only indexes 1–2; the leader has 1–5. The first `AppendEntries` (`prevLogIndex=4`) is rejected because D has nothing at index 4. The leader backs up to `prevLogIndex=2`, which D *does* have and which matches, so it ships entries 3, 4, 5. No deletion needed — D was simply behind. The "extra entries" case (Follower X above) is the interesting one, because there Raft must *discard* committed-looking-but-actually-uncommitted work. That's safe precisely because those entries were never committed (never on a majority), so no client was ever told they succeeded.

Best practice: Decrementing `nextIndex` by one per round-trip is correct but slow when a follower is far behind (one RPC per missing entry). Production systems use the paper's optimization: the rejected follower returns the *term of the conflicting entry and the first index it has for that term*, letting the leader skip a whole term's worth of indexes in one jump. etcd and others implement this. Don't ship the naive one-at-a-time version under load.

4.3 Commitment — the majority rule and the Figure-8 subtlety

An entry is **committed** the moment it is safely stored on a **majority** of servers. Once committed, it is durable: even if any minority of servers crash, every future leader will still contain it (you'll see why in §4.4). After committing, the leader advances its `commitIndex`, applies the entry, replies to the client, and piggybacks the new `commitIndex` on the next `AppendEntries` via `LeaderCommit` so followers apply it too.

```
COMMIT BY MAJORITY (5 servers, majority = 3). Leader appends entry @ index 5.
```

```
t0 Client --"set x=0"--> Leader
t1 Leader appends [3|x=0]@5 to its OWN log          (stored on 1 of 5)
t2 Leader --AppendEntries(entries=[3|x=0])--> A,B,C,D
    A: stored, ack --> 2 of 5
    B: stored, ack --> 3 of 5 *** MAJORITY reached ***
t3 Leader sees 3/5 -> commitIndex = 5 -> APPLY x=0 -> reply "OK" to client
t4 (C and D ack later; they catch up via leaderCommit on next heartbeat)
```

The entry is COMMITTED at t2 even though C and D haven't stored it yet. Durability comes from the majority overlap, not from "everyone".

Why a leader must not commit an old-term entry just by counting replicas (the Figure-8 problem)

Here is the famous subtlety. Suppose a new leader finds an entry from an *earlier* term sitting in its log, replicated on a majority. It is tempting to say "it's on a majority, therefore it's committed." **This is wrong, and it can lose committed data.** The danger: an entry can be on a majority of servers at one moment, then a *different* server with a shorter log can still win a future election and overwrite that entry — because elections compare logs by *term first*, not by length (see §4.4). So "on a majority right now" does not by itself mean "permanent."

Raft's fix is a rule called the **commitment restriction**:

*A leader only counts replicas to commit entries from its **own current term**. Entries from earlier terms are committed only indirectly — once an entry from the current term that sits after them gets committed, the Log Matching Property drags the older entries along with it.*

So a fresh leader does not "re-commit" what it inherited. It commits something new in its own term; the moment that new entry is on a majority and committed, everything before it is committed too, safely.

Example (Figure 8, simplified): Entry **E** was created by leader S1 in term 2 and copied to a majority — but *not yet committed*. S1 crashes. S5 (which never had **E**) wins term 3 because its log isn't behind on term. S5 crashes; S1 comes back as leader in term 4 and finishes spreading **E** to all. If S1 were allowed to call **E** "committed" just because it's now on a majority, disaster looms: S5 could come back, win again, and overwrite **E** — erasing an entry we'd called committed. Raft forbids this by never committing **E** (a term-2 entry) via replica-counting in term 4. **E** only becomes committed when S1 commits a fresh term-4 entry after it; at that point S5 can no longer win, so **E** is truly safe.

Common mistake: Believing "stored on a majority = committed, always." It is only true for an entry created in the *current* leader's term. For inherited older-term entries, on-a-majority is necessary but not sufficient — they become safe only once a current-term entry on top of them commits.

4.4 Safety — how Raft guarantees committed entries are never lost

Everything above only works if Raft can promise one thing: **a committed entry is never lost, never overwritten, and never appears at a different index later**. That promise is the **State Machine Safety Property**: *if any server has applied an entry at a given index, no other server will ever apply a different entry at that index*. Raft reaches it through two smaller guarantees.

(1) The Election Restriction — only an up-to-date candidate can win

When a candidate asks for a vote via **RequestVote**, it includes its **lastLogIndex** and **lastLogTerm**. A follower grants its vote **only if the candidate's log is at least as up-to-date as its own**. "Up-to-date" is compared like this:

1. Compare the **last log term** of both. The higher term wins — it is more up-to-date.
2. If the last terms are *equal*, the **longer log** (higher last index) wins.

Because a candidate needs a majority of votes, and any majority overlaps the majority that stored a committed entry, at least one voter has that committed entry. That voter will refuse to vote for any candidate whose log is missing it (such a candidate would be "less up-to-date"). Therefore a candidate that lacks a committed entry can *never* assemble a majority. It cannot win.

Analogy: Imagine electing a new editor for the team notebook, and the rule is "you may only vote for a candidate whose notebook is at least as complete as yours." Any officially-agreed line is in a majority of notebooks. To win, a candidate needs a majority of votes — but at least one of those voters holds every agreed line, and that person won't endorse anyone missing one. So the new editor always already has every agreed line. Nothing agreed can be lost across a handover.

(2) Leader Completeness → State Machine Safety

The Election Restriction gives the **Leader Completeness Property**: *every entry committed in some term is present in the logs of all leaders of higher terms*. In plain words: once an entry is committed, every future leader already has it. Combine that with the rule that a leader *never deletes or overwrites its own entries* (it only appends), and you get the conclusion: a committed entry sits at the same index in every leader from now on, gets copied to all followers, and is applied identically everywhere. That is exactly State Machine Safety.

Key takeaway: The safety argument is one clean chain — *majority overlap* means at least one voter holds each committed entry; the *Election Restriction* means that voter blocks any leader missing it; so *every future leader is complete*; and append-only logs mean a committed entry can never be displaced. Committed = permanent.

4.5 Cluster membership changes

Real clusters grow, shrink, and replace dead machines. The **configuration** is the set of servers that count toward a majority. Changing it is dangerous because for a brief moment the cluster might disagree about *who the members are*, and that can produce **two disjoint majorities electing two leaders at once** — a split brain, the exact thing consensus exists to prevent.

WHY A NAIVE SWITCH IS UNSAFE: growing from 3 servers to 5 in one step.

```
Old config C-old = {S1,S2,S3}    majority = 2
New config C-new = {S1,S2,S3,S4,S5} majority = 3
```

If servers adopt the new config at different times, you can get:

```
S1,S2 still think config is C-old -> {S1,S2} = majority of C-old
S3,S4,S5 think config is C-new    -> {S3,S4,S5} = majority of C-new
```

```
TWO majorities, no overlap -> S1 and S3 can BOTH win elections
                           -> TWO leaders in the same term. Split brain.
```

Joint consensus (C-old,new) — the general, safe method

Raft's original answer is a two-phase switch through a transitional combined configuration written **C-old,new**. The trick: while in this transitional state, a decision (winning an election, committing an entry) requires *separate* majorities from **both** the old set and the new set — not one merged majority. This makes overlapping majorities impossible, so no two leaders can co-exist. The steps:

1. The leader appends a special configuration entry `C-old,new` to its log and replicates it. The instant a server *stores* this entry (even before it's committed) it starts using it for all future decisions.
2. While `C-old,new` is in force, every election and every commit needs a majority of C-old *and* a majority of C-new. Neither old nor new can act alone.
3. Once `C-old,new` is committed, the leader appends a final `C-new` entry. After *that* commits, the transition is done; servers not in C-new shut down.

The overlap at each hop (old ↔ joint, joint ↔ new) guarantees a single decision-maker throughout. Clients keep being served during the change — there is no downtime.

Single-server-at-a-time — the simpler method

Joint consensus is fiddly to implement, so Ongaro's dissertation describes a simpler approach that most systems use: **add or remove only one server per configuration change**. Adding or removing a single server cannot create two disjoint majorities — the old and new majorities are guaranteed to overlap (the math: changing the count by one can't split the cluster into two non-overlapping majority halves). To go from 3 to 5, you do two single-server changes in sequence. This is what etcd and Consul use in practice.

Best practice: When adding a brand-new, empty server, bring it in as a **non-voting learner** first. It catches up on the log via `AppendEntries` without counting toward any majority. Only promote it to a full voter once its log is current. Otherwise a slow, empty new member can stall commits (the leader can't reach a majority that includes a member with nothing) or even trigger spurious leader changes. etcd's "learner" and Consul's non-voting members exist for exactly this.

Analogy: Changing a board of directors mid-meeting. If half the room thinks Alice was just removed and half thinks she's still voting, you can ratify two contradictory motions. Joint consensus is like requiring, during the handover, that a motion pass under *both* the old bylaws and the new ones simultaneously — so it's impossible to slip two conflicting decisions through. The single-at-a-time method is the conservative rule "only seat or unseat one director per meeting," which never produces two competing majorities in the first place.

4.6 Log compaction & snapshotting

The log grows forever — every command is a new entry. Left unchecked it would fill the disk and make restarts (which replay the whole log) unbearably slow. The fix is a **snapshot**: a compact dump of the state machine's *current state* at a point in time. Once you have a snapshot up to index N, you can throw away log entries 1...N, because their combined effect is already captured in the snapshot.

Each server snapshots independently. A snapshot records the state plus two pieces of bookkeeping: `lastIncludedIndex` (the last log index it covers) and `lastIncludedTerm` (that entry's term) — kept so the Log Matching consistency check still works at the boundary after the entries are gone.

This creates a corner case: what if a follower is so far behind that the leader has already discarded the entries that follower still needs? The leader can't send entries it no longer has. So Raft adds a third RPC, `InstallSnapshot`: the leader ships its whole snapshot to the lagging follower, who installs it wholesale (discarding its own now-superseded log) and resumes normal `AppendEntries` from the snapshot's end. Its fields: `term`, `leaderId`, `lastIncludedIndex`, `lastIncludedTerm`, `offset` + `data` (the snapshot bytes, sent in chunks for large states), and `done` (last chunk flag).

Example: A 5-server cluster has processed a million commands. Each server periodically snapshots — say at index 950,000 — and deletes entries 1...950,000, keeping only the recent tail. A follower that was offline for an hour comes back asking for index 200,000, which no longer exists anywhere. The leader sends an `InstallSnapshot` with `lastIncludedIndex = 950,000`; the follower replaces its stale log with that snapshot in one shot, then catches up the last few thousand entries normally — far faster than replaying a million.

Common mistake: Treating snapshotting as an optional add-on. Without it, a long-running Raft cluster's restart time and disk use grow without bound, and a briefly-offline follower can take a very long time to rejoin by re-streaming the entire history. Snapshotting is mandatory for any production deployment, not a nice-to-have.

Common mistakes & misconceptions

Common mistake: Thinking the consistency check compares whole logs. It checks a *single* entry at `prevLogIndex`. The Log Matching Property is what makes that one check imply the entire prefix matches.

Common mistake: Assuming "replicated on a majority" always means "committed." It only does for entries in the leader's *current* term. Counting replicas to commit an older-term entry directly is the Figure-8 bug and can lose data.

Common mistake: Believing the longest log always wins an election. Raft compares the *last term first*, and only uses length as a tie-breaker when last terms are equal. A shorter log with a higher last term is "more up-to-date."

Common mistake: Changing cluster membership by just editing a config file and restarting nodes. Without joint consensus or single-at-a-time changes, you can create two disjoint majorities and elect two leaders (split brain).

Common mistake: Thinking a follower's discarded "extra" entries were a bug or data loss. Those uncommitted entries were never on a majority and no client was ever told they succeeded, so deleting them is correct and loses nothing the system promised.

Best practices

Best practice: Use the fast log-backtracking optimization (return conflicting term + first index of that term on a rejection) instead of decrementing `nextIndex` by one per RPC. It turns slow per-entry repair into per-term jumps.

Best practice: Add new members as non-voting learners and let them catch up before promoting them to voters, so an empty newcomer never stalls commits or destabilizes elections.

Best practice: Change membership one server at a time (or use proper joint consensus). Never do a multi-server swap in a single step.

Best practice: Snapshot regularly and tune the trigger (e.g. every N entries or M bytes). Keep `lastIncludedIndex` / `lastIncludedTerm` so the boundary consistency check still works, and stream large snapshots in chunks.

Best practice: Persist `currentTerm`, the vote, and the log to stable storage *before* responding to any RPC. Acknowledging an entry you haven't durably stored breaks the majority-durability guarantee on a crash.

Section summary

- `AppendEntries` carries `prevLogIndex` / `prevLogTerm` as a fingerprint; a follower accepts new entries only if that fingerprint matches, which enforces the **Log Matching Property** (same index+term $\Rightarrow$ identical logs up to there).
- A divergent follower is repaired by the leader backing up `nextIndex` until the logs agree, then overwriting the follower's conflicting tail with its own entries.
- An entry is **committed** once stored on a **majority**; the leader then advances `commitIndex` and tells followers via `leaderCommit`.
- A leader commits by replica-count only for entries in its **own term**; older entries commit indirectly once a current-term entry on top of them commits — this is the Figure-8 fix.
- The **Election Restriction** (vote only for an at-least-as-up-to-date log, compared by last term then last index) plus majority overlap yields **Leader Completeness** and therefore **State Machine Safety**: committed entries are never lost or reordered.

- Naive membership changes can create two majorities and split brain; Raft prevents this with **joint consensus (C-old,new)** or the simpler **single-server-at-a-time** method.
- Logs can't grow forever; **snapshots** compact committed state, and **InstallSnapshot** brings a far-behind follower up to date when needed entries have been discarded.
- Across the protocol, "committed" is an absolute promise — once an entry is committed, every future leader has it and every server eventually applies it at the same index.

5. Paxos — The Original Consensus Algorithm

This is the algorithm that started it all. **Paxos** is a protocol that lets a group of unreliable machines agree on a single value, even when some of them crash, messages are lost or delayed, and several machines try to propose different values at the same time. "Consensus" here means exactly that: *getting a set of nodes to agree on one value and never change their minds*.

Paxos is famous for two opposite things at once: it is provably correct, and it is notoriously hard to understand. We are going to teach it slowly, one piece at a time. By the end you will see that the core idea is actually small — it is just packed very tightly.

5.1 History and why Paxos still matters

Leslie Lamport (the same person who gave us logical clocks and happens-before) first described Paxos in a 1990 paper called "*The Part-Time Parliament*." He wrote it as a joke story about a fictional Greek island called Paxos, whose parliament passed laws even though legislators kept wandering in and out of the chamber (the "parliamentarians" are the unreliable machines). The joke backfired: people found the story so confusing that almost nobody understood the algorithm hidden inside it. The paper sat largely ignored for years.

In 2001 Lamport gave up on the joke and published "*Paxos Made Simple*," a short, plain-English re-explanation. Its famous opening line is roughly: "The Paxos algorithm, when presented in plain English, is very simple." Many readers disagreed about the "simple" part — which is exactly why the Raft paper (the next section) was written, with the explicit goal of being *understandable*. But Paxos still matters enormously, because it is the theoretical bedrock. Google's Chubby lock service and Spanner database, Microsoft's Azure storage, Amazon's internal systems, and many others are built on Paxos or close relatives of it. Even systems that use Raft or ZAB instead are using ideas that Paxos established and proved correct first.

Key takeaway: Paxos is the original, mathematically-proven solution to consensus. You may never implement raw Paxos yourself, but every modern consensus system is a descendant of it. Understanding Paxos is understanding *why* consensus is possible at all.

5.2 The setup: roles and the goal

Before any steps, we need the cast of characters. Paxos splits the work into three **roles**. A "role" is just a job a machine performs — and importantly, *one physical machine can play several roles at once*. In real deployments every server is usually a proposer, an acceptor, and a learner all together. We separate the roles only to explain the logic cleanly.

Role	Plain-English job	What it does
Proposer	The one who suggests a value	Tries to get a value chosen. Drives the whole protocol by sending requests.
Acceptor	The fault-tolerant memory / voter	Votes on proposals and <i>remembers</i> what it has promised and accepted, even across crashes. The acceptors are the system's persistent memory.
Learner	The one who finds out the result	Learns which value was finally chosen, so it can act on it (e.g. apply it to a database).

The **goal of single-decree Paxos** ("single-decree" = deciding *one* value, one single decision) is precise:

- **Safety:** Only a value that was actually proposed can be chosen. Only *one* value is ever chosen. Once a value is chosen, it is chosen forever — nobody can un-choose it or choose a different one. A learner never learns a value unless it really was chosen.
- **Liveness (best-effort):** If things stabilize, some proposed value eventually does get chosen and learners find out. (As we will see, Paxos cannot *guarantee* this in a fully asynchronous world — that is the FLP result from the foundations guide — so it settles for "eventually, once the network behaves.")

The mechanism that makes a value "chosen" is a **majority** of acceptors. A majority means more than half (for 5 acceptors, that is 3). The magic property of majorities is that *any two majorities must overlap in at least one acceptor*. Two groups of 3 chosen from 5 must share at least one member. That single overlapping acceptor is the secret that makes Paxos safe — keep it in mind, we will use it.

Analogy: Think of the acceptors as a row of notaries. A proposer is a lawyer trying to get a contract notarized. The contract counts as "official" only when a majority of notaries have stamped the *same* contract. Because any two majorities of notaries share at least one person, and a notary refuses to contradict a stamp they already gave, you can never end up with two different "official" contracts.

5.3 Proposal numbers (ballots): giving every attempt a unique, ordered ID

Several proposers may be active at once, and a single proposer may retry after a failure. To keep order out of this chaos, every proposal attempt is tagged with a **proposal number** (also called a *ballot number*). Two rules govern these numbers:

1. **They are totally ordered** — given any two, you can always say which is bigger. Bigger means "newer / takes priority."
2. **They are globally unique** — no two proposers ever pick the same number. This is essential: if two proposals shared a number, the tie-breaking that keeps Paxos safe would fall apart.

How do you make numbers unique across machines with no shared clock? The standard trick is to build each number out of two parts: a locally-increasing counter combined with the server's own unique ID — for example the pair `(counter, serverId)`, compared counter-first and breaking ties by `serverId`. Server A's numbers might be 1.A, 2.A, 3.A while server B's are 1.B, 2.B, 3.B; they can never collide because the `serverId` differs. A simpler-to-picture version: give proposer A only odd numbers and proposer B only even numbers. Either way, each proposer can always pick a number higher than any it has seen, and no two numbers are ever equal.

Common mistake: Thinking the proposal number is the *value* being agreed on. It is not. The number is just a priority tag on an *attempt*. The value is separate data (e.g. "set x = 7"). One value can be carried by many proposals with different numbers; the number decides whose attempt wins, not what gets stored.

5.4 Phase 1 — Prepare / Promise

Paxos runs in two phases. Phase 1 is about *claiming priority and discovering history*. The proposer is essentially asking, "Can I have a turn at number `n`, and by the way, has anything already been agreed that I need to honor?"

Phase 1a — Prepare (proposer → acceptors). The proposer picks a brand-new proposal number `n` (higher than any it has used) and sends a `Prepare(n)` message to at least a majority of acceptors. Note: at this point it sends *no value yet* — only the number.

Phase 1b — Promise (acceptor → proposer). When an acceptor receives `Prepare(n)`:

- If `n` is greater than every proposal number this acceptor has already responded to, it replies with a **Promise**. The Promise is a binding commitment: "*I promise I will never again accept any proposal numbered less than `n`.*" It is effectively raising the floor.
- Crucially, along with the promise the acceptor reports back **any value it has already accepted** — specifically the value, tagged with the proposal number under which it accepted it: it returns `(accepted_number, accepted_value)`, or "nothing" if it has accepted no value yet. This is how already-agreed history travels forward.
- If `n` is *not* higher than something it has already promised, the acceptor ignores the request (or sends a "no" / NACK so the proposer can give up faster).
- The acceptor must **write this promise to stable storage** (disk) before replying, so that if it crashes and restarts it still remembers the floor it raised. The acceptors are the durable memory of the system.

PHASE 1: Prepare / Promise (proposer P, acceptors A1 A2 A3)

```
P           A1           A2           A3
|---Prepare(n)--->|           |           |
|---Prepare(n)----->|           |           |
|---Prepare(n)----->|           |           |
|           |           |           |
|   (each acceptor: is n > my highest seen number?)
|   (A1,A2,A3 had accepted nothing yet -> promise, report "none")
|           |           |           |
|<--Promise(n, none)|           |           |
|<--Promise(n, none)-----|           |           |
|<--Promise(n, none)-----|           |           |
|
|   P heard from a MAJORITY (3 of 3). It may proceed to Phase 2.
|   No prior value reported -> P is free to use its OWN value.
```

5.5 Phase 2 — Accept / Accepted

Phase 2 is where an actual value gets pinned down. The proposer now knows it has the floor (a majority promised) and knows the relevant history (whatever values came back in the promises).

Phase 2a — Accept (proposer → acceptors). If the proposer received Promises from a *majority* of acceptors, it now chooses which value to push, using one strict rule:

- Look at all the Promises that came back. If *any* of them reported an already-accepted value, the proposer **must** use the value attached to the *highest accepted_number* among them. It does **not** get to use its own value — it is forced to carry forward the most recent already-accepted value.
- Only if *none* of the majority reported any accepted value is the proposer free to propose its *own* value.

Call the chosen value `v`. The proposer sends `Accept(n, v)` to the acceptors.

Phase 2b — Accepted (acceptor → proposer/learners). When an acceptor receives `Accept(n, v)`, it accepts it *unless* it has, in the meantime, already promised some higher number than `n`. In plain terms: "I'll accept this as long as nobody newer than you came along while you were gone." On accepting, it records `(n, v)` durably and notifies the learners. When a **majority** of acceptors have accepted `(n, v)`, the value `v` is **chosen** — permanently. Learners that hear a majority of `Accepted` messages know the decision and can act on it.

PHASE 2: Accept / Accepted (continuing from Phase 1)

P	A1	A2	A3
P picks v = its own value (no prior value was reported).			
---Accept(n, v)->			
---Accept(n, v)----->			
---Accept(n, v)----->			
(each: have I promised anything > n ? No -> accept (n, v), save to disk)			
<--Accepted(n, v)-			
<--Accepted(n, v)-----			
<--Accepted(n, v)-----			
MAJORITY accepted (n, v) ==> v is CHOSEN, forever.			
Learners that see a majority of Accepted(n, v) now know: the answer is v .			

Example: Five acceptors A1–A5. Proposer P (a config server) wants to set the cluster leader to "node-7".

- (1) P sends `Prepare(5.P)` to all five. A1, A2, A3 reply `Promise(5.P, none)` — that is a majority of 3, none has accepted anything.
- (2) Since no prior value came back, P is free to use its own value, so it sends `Accept(5.P, "node-7")`.
- (3) A1, A2, A3 had promised nothing higher than 5.P, so they accept and persist `(5.P, "node-7")` and tell the learners. Three of five accepted — a majority — so "node-7" is now the chosen leader, irrevocably, even if P crashes one millisecond later.

5.6 Why Paxos is SAFE — the heart of the algorithm

This is the part worth reading twice. Everything above exists to make one guarantee true: **once a value could possibly have been chosen, every future proposal is forced to propose that same value.**

So the system can never choose two different values. Here is the chain of reasoning, built from two facts you already have.

Fact 1 — Majorities overlap. Suppose value v was chosen at proposal number n . By definition that means a majority of acceptors — call it set $M1$ — accepted (n, v) .

Fact 2 — A later proposal must talk to a majority too. Now some proposer comes along later with a higher number n' (where $n' > n$). To get anywhere it must complete Phase 1: it must collect Promises from *its own* majority — call it set $M2$. Because any two majorities overlap, $M1$ and $M2$ share **at least one acceptor**. Call that shared acceptor x .

The punchline. Acceptor x is in $M1$, so it accepted (n, v) . When the later proposer asks x in Phase 1, x reports back the value it accepted — (n, v) — inside its Promise. Now the Phase 2a rule kicks in: the later proposer is *forced* to use the highest-numbered accepted value it heard about. x just told it about v at number n . So the later proposer cannot use a value of its own — it **must** re-propose v . Therefore proposal n' also proposes v . The same argument applies to n'' , n''' ,

and so on forever. Every proposal numbered higher than `n` proposes `v`. No second value can ever be chosen.

The subtle bit that closes the last hole: how can the later proposer be sure it learned about a chosen value, if maybe only some of `M1` had accepted by then? The answer is the *promise*. When `x` promised not to accept anything below `n'`, it also locked the door: any proposal between `n` and `n'` can no longer slip a different value past `x`. So either the later proposer sees `v` directly, or no competing value could have reached a majority through `x` in the first place. The "promise" (raise the floor) and the "adopt the highest accepted value" rule work as a pair: the promise blocks the past from changing, and the adopt-rule forces the future to inherit it.

Key takeaway: Paxos safety = *majority overlap* + *acceptors remember* + *every new proposer must adopt the highest already-accepted value it learns about*. Together these guarantee that the very first value that reaches a majority becomes the permanent decision, and no later, higher-numbered proposal can override it.

Common mistake: Believing the proposer with the highest number "wins" by getting its preferred value chosen. A higher number wins the right to *drive the protocol*, but it usually does **not** get to install its own value — if a value was already accepted anywhere in its majority, it is forced to carry that existing value forward. Higher number = right to proceed, not right to overwrite.

5.7 The liveness problem — dueling proposers and FLP

Paxos is always safe, but it is *not* always guaranteed to finish. The classic failure mode is **dueling proposers** (also called livelock — the system is busy but makes no progress). Picture two proposers, P1 and P2, each retrying with ever-higher numbers, perfectly out of step:

```
DUELING PROPOSERS (livelock)    P1 and P2 over acceptors A1 A2 A3

P1: Prepare(1) -----> A1 A2 A3  promise(1)
P2:   Prepare(2) -----> A1 A2 A3  promise(2)  (raises floor to 2)
P1: Accept(1, x) -----> A1 A2 A3  REJECT  (already promised 2 > 1)
P1:   Prepare(3) -----> A1 A2 A3  promise(3)  (raises floor to 3)
P2: Accept(2, y) -----> A1 A2 A3  REJECT  (already promised 3 > 2)
P2:   Prepare(4) --> A1 A2 A3  promise(4)
P1: Accept(3, x) -----> A1 A2 A3  REJECT  (already promised 4 > 3)
... and so on, forever. Each Prepare invalidates the other's pending Accept.
No value is ever chosen, even though nothing crashed.
```

Each proposer's Phase 1 keeps raising the floor and knocking out the other's Phase 2 before it can complete. Nobody crashed, no message was lost — yet no progress is made. This is not a bug in Paxos; it is the **FLP impossibility result** from the foundations guide showing up in real life: in a fully asynchronous system (no bound on message delays, no perfect failure detection), *no* consensus

algorithm can guarantee it always terminates. Paxos chooses to sacrifice guaranteed liveness to keep absolute safety. Safety is never violated; only progress can stall.

The standard fix has two parts:

- **Elect a single distinguished proposer (a "leader").** Agree — informally, by a weak election that doesn't need to be perfect — that only one proposer normally issues proposals. With just one proposer, there is nobody to duel with, so Phase 1 and Phase 2 complete cleanly. Safety does not depend on the election being correct: if two leaders accidentally exist, Paxos is still *safe* (two values can never be chosen) — you only risk a temporary stall.
- **Randomized / exponential backoff.** When a proposer's attempt is rejected, it waits a *random* amount of time before retrying (and the wait grows on repeated failures). Randomness desynchronizes the duelists so one of them gets a clean window to finish both phases. This is the same trick Ethernet and Raft elections use to break symmetric ties.

A common optimization, **Multi-Paxos**, takes the leader idea further: a stable leader runs Phase 1 just *once* to claim a range of decisions, then only does the cheap Phase 2 for each new value. This makes the steady state one round-trip per decision — and it is essentially the shape Raft formalizes with a strong leader.

Analogy: Two people reach for the same door handle, both pull back politely, then both reach again — repeating forever and never getting through. That's dueling proposers. The fix is the same as in real life: appoint one person to go first (a leader), and have anyone who collides wait a *random* moment before trying again so they stop colliding in lockstep.

Common mistakes & misconceptions

Common mistake: Confusing the proposal number with the value. The number is a priority tag on an *attempt*; the value is the separate data being agreed. Many proposals with different numbers can carry the very same value.

Common mistake: Thinking the highest-numbered proposer always installs its own value. If a value was already accepted in its majority, the protocol *forces* it to re-propose that existing value. The high number only grants the right to proceed, not to overwrite.

Common mistake: Assuming Paxos always terminates. Dueling proposers can livelock forever (FLP in action). Paxos guarantees *safety* always, but *liveness* only under partial synchrony plus a leader and backoff.

Common mistake: Believing acceptors can be stateless or in-memory only. Acceptors are the durable memory of the system — every promise and acceptance must be written to stable storage *before* replying, or a crash-and-restart can break safety.

Common mistake: Thinking basic Paxos decides a whole log of commands. Single-decree Paxos decides exactly *one* value. Replicating a sequence of commands needs Multi-Paxos (one Paxos instance per log slot), which is a separate, larger construction.

Common mistake: Treating two simultaneous leaders as a correctness bug. Two leaders only threaten *progress* (a stall), never *safety* — Paxos can never choose two different values regardless of how many proposers run at once.

Best practices

Best practice: Always run with an odd number of acceptors (3, 5, 7). It maximizes fault tolerance per node — 5 acceptors tolerate 2 failures and still form a majority — and avoids ambiguous even-split majorities.

Best practice: Use a single distinguished leader proposer in steady state (Multi-Paxos style). It eliminates dueling, collapses the steady state to one round-trip per decision, and keeps the system live.

Best practice: Make proposal numbers unique by construction — pair a local counter with the server ID, e.g. `(counter, serverId)`. Never rely on wall-clock time alone for uniqueness or ordering.

Best practice: Persist promises and acceptances to disk *before* sending the reply (write-ahead). The acceptors are the system's memory; a reply that isn't yet durable can be lost in a crash and violate safety.

Best practice: Add randomized/exponential backoff on every rejected proposal so that colliding proposers desynchronize instead of dueling in lockstep.

Best practice: Unless you have a strong reason, build on a battle-tested implementation (etcd/Raft, ZooKeeper/ZAB, a vetted Paxos library) rather than hand-rolling Paxos — the edge cases around durability, leader change, and message reordering are exactly where subtle safety bugs hide.

Section summary

- Paxos, from Lamport's *Part-Time Parliament* (1990) and *Paxos Made Simple* (2001), is the original, provably-correct consensus algorithm and the ancestor of nearly every modern one.
- It uses three roles — **proposer** (suggests), **acceptor** (votes and remembers durably), **learner** (finds out the result) — and one machine can play all three.
- The goal of single-decree Paxos is to get a *majority* of acceptors to agree on exactly one value, permanently and safely.
- Every attempt carries a unique, totally-ordered **proposal number**, typically built as `(counter, serverId)` so no two ever collide.
- **Phase 1 (Prepare/Promise)**: proposer sends `Prepare(n)`; an acceptor promises to reject anything below `n` and reports back any value it already accepted.
- **Phase 2 (Accept/Accepted)**: with a majority of promises, the proposer sends `Accept(n, v)`, where `v` is the highest already-accepted value it learned about — or its own value if there was none; acceptors accept unless they've promised something higher. A majority of accepts = chosen forever.
- Safety rests on three things together: *majorities always overlap*, *acceptors remember*, and *new proposers must adopt the highest already-accepted value* — so the first value to reach a majority can never be overridden.
- Paxos can **livelock** with dueling proposers (the FLP impossibility made concrete); it sacrifices guaranteed liveness, never safety.
- The fix is a single **distinguished/leader proposer** plus **randomized backoff**; Multi-Paxos extends this to an efficient one-round-trip-per-decision log, the shape Raft later formalized.

6. Multi-Paxos, Raft vs Paxos & the Real World

So far you have seen consensus as a way to agree on *one* value. In the previous sections, basic Paxos (single-decree Paxos) agrees on a single decision: one slot, one chosen value, done. But real systems almost never want to agree on just one thing. A database wants to agree on a never-ending **stream** of operations: "set x=5", then "delete y", then "append to z", in a strict order that every replica applies identically. This section is about how the theory turns into working systems, and how you choose between them.

Let me define the central idea up front. A **replicated log** is an ordered list of commands, numbered 1, 2, 3, 4, ..., where every replica stores the *same* commands in the *same* order. Each numbered position is called a **log slot** (or log index). A **state machine** is just your application: it starts in some state and changes state by applying commands one at a time. The whole trick of practical consensus is called **state machine replication (SMR)**: if every replica starts in the same state and applies the exact same sequence of commands in the exact same order, then every replica ends up in the exact same state. Consensus is the tool that makes all replicas agree on "what is the command in slot 1? slot 2? slot 3?" — agreeing on the log makes the whole system behave like one reliable machine.

Analogy: Think of a group of accountants, each keeping their own copy of the same ledger. If they all write down the exact same transactions in the exact same order, all their ledgers will match at the end — even if they never compare totals. Consensus is the rule that decides "what is transaction #1, #2, #3" so that nobody writes a different line on the same row. The state machine is the running balance; the log is the list of transactions.

6.1 From single-decree to a LOG: Multi-Paxos

What it is. **Multi-Paxos** is simply: run a separate instance of basic Paxos for *each* log slot. Slot 1 has its own Paxos run, slot 2 has its own Paxos run, and so on. Each slot independently chooses one value (one command), and once chosen it never changes. Stack the chosen values in slot order and you have your replicated log.

The problem it solves. Basic Paxos agrees on one value with two phases of message round-trips. If you naively ran full Paxos for every command, every single command would pay that full cost. That is wasteful, and it is also dangerous: if many nodes try to be proposer at the same time, they can keep interrupting each other with higher proposal numbers ("dueling proposers") and make no progress. Multi-Paxos fixes both with one idea: **elect a stable leader and reuse it**.

Quick recap of basic Paxos phases (covered in earlier sections — restated only so the optimization makes sense):

- **Phase 1 — Prepare / Promise.** A proposer picks a **proposal number** (also called a *ballot number* — a unique, ever-increasing tag that says "how recent this attempt is") and sends **Prepare(n)** to the acceptors. An acceptor that has not seen anything higher replies **Promise**: "I won't accept

anything older than `n`, and here is the most recent value I already accepted, if any." This phase is really about *winning the right to propose and learning any value that might already be chosen*.

- **Phase 2 — Accept / Accepted.** The proposer sends `Accept(n, value)`; a majority replying `Accepted` means the value is chosen.

How the stable-leader optimization works, step by step. Here is the key insight: Phase 1 does not mention any specific value or slot. It just establishes "I, proposer with ballot `n`, am now in charge, and here is what's already been accepted." So you can run Phase 1 **once** for an *entire range of future slots*, not per command.

1. A node decides to become leader. It runs Phase 1 (`Prepare`) once with a ballot number `n`, but for *all log slots from "next empty slot" onward*. A majority of acceptors promise.
2. Now this leader is established. For each new client command, it simply assigns the next free slot and sends only **Phase 2** (`Accept(n, slot, command)`) to the acceptors. One round trip. No Phase 1.
3. This continues for command after command — only Phase 2 each time — as long as the leader stays alive and no other node challenges it with a higher ballot.
4. If the leader crashes or is suspected dead, some other node runs Phase 1 again with a higher ballot `n+1`, becomes the new leader, learns any half-finished slots from the promises, and resumes single-round-trip operation.

Why it matters. In the common, steady-state case (a healthy stable leader), committing a command costs just **one round trip to a majority** — roughly two message delays — instead of two full phases (about four message delays). That is nearly half the latency, and it removes dueling proposers because there is normally only one active proposer. Phase 1 becomes a rare event that happens only on leader change.

SINGLE-DECREE PAXOS (full protocol, every value pays for both phases)

Proposer	Acceptors (majority)	
Prepare(n) ----->		Phase 1
<----- Promise ----		(win leadership + learn)
Accept(n,v) ----->		Phase 2
<----- Accepted ----		(value chosen)
=> ~4 message delays PER VALUE		

MULTI-PAXOS WITH A STABLE LEADER (Phase 1 done once, then reused)

Leader	Acceptors (majority)	
Prepare(n) ----->		Phase 1 — ONCE, for all future slots
<----- Promise ----		
----- leader is now stable		
Accept(n,slot1,A)->		Phase 2 only (1 round trip) -> slot 1 = A
<---- Accepted ----		
Accept(n,slot2,B)->		Phase 2 only (1 round trip) -> slot 2 = B
<---- Accepted ----		
Accept(n,slot3,C)->		Phase 2 only (1 round trip) -> slot 3 = C
=> ~2 message delays PER VALUE after the one-time setup		

Example: A 3-node key-value store using Multi-Paxos. Node A wins Phase 1 with ballot 10 and becomes leader. A client sends "SET balance=100". A assigns it slot 1 and sends `Accept(10, slot1, "SET balance=100")`; B and C reply `Accepted` — that is a majority (2 of 3), so slot 1 is committed in one round trip. Next "WITHDRAW 30" goes into slot 2, again one round trip. A never re-runs Phase 1. If A then crashes, B runs `Prepare(11)` for slots from 3 onward, learns from the promises that slots 1 and 2 are already committed, fills any gaps, and continues as the new leader.

Common mistake: Thinking "Multi-Paxos" is a single, fully specified protocol you can copy from a paper. It is not. Lamport's "Paxos Made Simple" describes the leader/log idea in only a paragraph or two and leaves out the hard practical parts: how leader election actually works, how you fill gaps in the log, how you handle a node that fell behind, how you change membership. Every production Multi-Paxos system had to invent and verify these details itself — which is exactly the pain that motivated Raft.

6.2 Raft vs Paxos — a fair comparison

Raft (Diego Ongaro and John Ousterhout, 2014, "In Search of an Understandable Consensus Algorithm") was designed with an unusual primary goal: *understandability*. It provides the *same* guarantees and the *same* efficiency as Multi-Paxos — the paper explicitly says Raft "produces a result equivalent to (multi-)Paxos, and it is as efficient as Paxos." The difference is structure. Raft starts from a strong leader and a single log, and deliberately reduces the number of moving states you have to reason about.

Two terms you need before the comparison:

- **Strong leader.** In Raft, log entries flow in *one direction only*: from the leader out to followers, never the other way. Clients talk to the leader; the leader is the single source of truth for log order. This is stricter than Multi-Paxos, where any node can in principle propose for a slot.
- **Term.** Raft's name for a ballot/era. A **term** is a numbered period of time with at most one leader. Terms increase monotonically; each leader election starts a new term. Terms play the same role as Paxos ballot numbers: they let nodes detect stale leaders.

Raft has exactly two RPCs (remote procedure calls — a function call made over the network from one node to another):

- **RequestVote** — sent by a candidate during an election. Key fields: **term** (the candidate's new term), **candidateId**, **lastLogIndex** and **lastLogTerm** (so voters only elect a candidate whose log is at least as up-to-date as their own — this is the safety rule that keeps committed entries safe).
- **AppendEntries** — sent by the leader to replicate log entries *and* as an empty heartbeat to keep its leadership alive. Key fields: **term**, **leaderId**, **prevLogIndex** and **prevLogTerm** (the entry immediately before the new ones, used as a consistency check so a follower only accepts entries that line up with what it already has), **entries[]** (the new commands), and **leaderCommit** (how far the leader has committed).

Notice the mapping: Raft's **RequestVote** $\approx$ Paxos Phase 1 (win leadership for a term), and Raft's **AppendEntries** $\approx$ Multi-Paxos Phase 2 (commit a command in one round trip). They are the same machine wearing different clothes.

RAFT NORMAL OPERATION (one term, stable leader)

```

Client      Leader (term 4)      Follower1  Follower2
| cmd ---> |                      |          |
|          | AppendEntries(term=4, prevLogIndex=2,
|          |   entries=[slot3:cmd], leaderCommit=2) -----> | |
|          | <----- success (matched prevLog) -----> | |
|          | majority acked => slot 3 committed
| <-- ok -| apply to state machine, tell followers via leaderCommit

```

RAFT LOG as boxes (followers must match the leader exactly):

```

index:   1       2       3       4
         +-----+-----+-----+-----+
leader  | t1 A   | t1 B   | t4 C   | t4 D   |
         +-----+-----+-----+-----+
(tN = term in which the entry was created; index = log slot)

```

Dimension	Multi-Paxos	Raft
Primary design goal	Provable correctness; minimal core	Understandability and ease of correct implementation
Guarantees / efficiency	Safe under crashes; ~1 round trip with stable leader	Identical safety and efficiency (paper says "equivalent to multi-Paxos, as efficient as Paxos")
Leader model	Optional optimization. Leader is a performance trick; any node may propose. Weaker, more flexible leader.	Mandatory <i>strong</i> leader. Entries flow leader → follower only. Simpler to reason about.
Log handling	Slots can be chosen out of order; gaps allowed and filled later. More flexible, more bookkeeping.	Log is contiguous and append-only; a follower's log must match the leader exactly up to a point (Log Matching Property). No gaps.
Era / freshness tag	Ballot (proposal) numbers	Terms (one leader per term)
How it's specified	Core proven in papers; practical Multi-Paxos (log, leader, recovery) under-specified — each team fills gaps	Fully specified end-to-end (election, replication, safety, membership, log compaction) in one paper with a formal proof and reference impls
Membership changes	Described abstractly (e.g. use a log entry to change the config); details vary by system	Built-in: <i>joint consensus</i> (overlap old+new config) or single-server add/remove. Spelled out.
Real implementations	Chubby, Spanner — built by expert teams; famously hard to get exactly right	etcd, Consul, CockroachDB/TiKV, Kafka KRaft — many independent, interoperable implementations
Flexibility ceiling	Higher — leaderless and disjoint-quorum variants (EPaxos, Flexible Paxos) extend the family	Lower by design — the strong-leader constraint trades flexibility for clarity

The honest summary: Raft and Multi-Paxos give you the *same* correctness and the *same* common-case speed. Paxos is the more general, more flexible family but is notoriously easy to get subtly wrong because the practical pieces are under-specified. Raft constrains the design (strong leader, contiguous log) to make a complete, teachable, implementable spec — which is why so many recent systems pick it. "Raft is better" is too strong; "Raft is easier to implement correctly, Paxos is more flexible" is the fair statement.

Common mistake: Believing Raft is faster or "more correct" than Paxos. It is neither. The authors deliberately matched Paxos's guarantees and performance; the only intended improvement was understandability. If someone claims a raw throughput win, they are comparing two specific implementations, not the algorithms.

6.3 Real-world systems and what they use consensus FOR

System	Algorithm	What it uses consensus for
etcd	Raft	A distributed key-value store for cluster configuration and coordination; it is the consistent store behind Kubernetes (all cluster state lives here).
HashiCorp Consul	Raft	Service discovery, health checks, and a consistent key-value store; Raft keeps the service catalog and config replicated.
CockroachDB / TiKV	Raft (per data range)	Distributed SQL databases. Data is split into ranges; each range is a small Raft group replicating its writes, so each shard agrees on its own ordered log.
Apache ZooKeeper	ZAB (Paxos-like atomic broadcast)	Coordination service: locks, leader election, config, naming. ZAB (ZooKeeper Atomic Broadcast) is a leader-based protocol — similar in spirit to Multi-Paxos — that orders all updates through one leader.
Google Chubby	Paxos	A lock service and small-file store used across Google for leader election and coordination; one of the first big production Paxos deployments.
Google Spanner	Paxos (per tablet/shard)	Globally distributed SQL database. Each data shard is replicated by its own Paxos group across data centers, giving consistent, ordered writes worldwide.
Apache Kafka (KRaft mode)	Raft	Kafka 3.0+ replaced its ZooKeeper dependency with KRaft (Kafka Raft) to manage cluster metadata — topics, partitions, broker membership — internally via a Raft log.

Notice a pattern shared by ZAB, ZooKeeper, etcd, Chubby, and Consul: these are **coordination services**. Applications rarely run consensus directly; instead they lean on one of these systems for the few things that truly need agreement (who is the leader, what is the current config, who holds this lock) and keep their high-volume data path outside consensus.

Analogy: Consensus is like the town's official notary. You do not notarize every email you send — that would be unbearably slow and expensive. You notarize only the handful of documents where everyone must later agree the exact wording and order: the deed, the contract, the will. etcd/ZooKeeper/Consul are the notary's office, shared by the whole town, so each application doesn't have to hire its own.

6.4 Beyond the basics (advanced — intuition only)

EPaxos — leaderless, commutativity-aware Paxos

Egalitarian Paxos (EPaxos) removes the single leader entirely. The motivation: a single leader is a bottleneck and a single point of slowness — clients far from the leader pay a long round trip. EPaxos lets *any* replica act as coordinator for a command. The clever part uses **commutativity**: two commands *commute* if running them in either order gives the same result (e.g. "set x=1" and "set y=2" don't interfere, but "set x=1" and "set x=2" do). When concurrent commands commute, EPaxos commits them in just **two message delays** (the "fast path") without ordering them relative to each other. Only *conflicting* commands need extra work to agree on an order (a slower, classic-Paxos-style path). The intuition: don't waste time ordering things whose order doesn't matter. The cost is complexity — tracking dependency sets between commands is intricate and has historically been hard to get right.

Flexible Paxos — quorum intersection, revisited

Classic Paxos uses majority quorums because any two majorities overlap in at least one node, and that overlap is what carries information forward safely. **Flexible Paxos** (Howard, Malkhi, Spiegelman, 2016) proved a sharper truth: the only intersection that actually matters is *between Phase 1 quorums and Phase 2 quorums* — not within a phase. So your Phase 1 (leader election) quorums and Phase 2 (commit) quorums must overlap each other, but two Phase 2 quorums need *not* overlap. Practically, you can shrink the commit quorum (faster, cheaper writes) at the price of a larger election quorum (rarer, so it's fine). The intuition: majorities were a convenient sufficient condition, not a necessary one — you can trade quorum sizes between the two phases.

Byzantine fault tolerance & PBFT — consensus when nodes can LIE

Everything above assumes **crash faults**: a node either follows the protocol correctly or stops. A **Byzantine fault** is worse — a faulty node can do *anything*, including lie, send different messages to different peers, or actively try to corrupt agreement (a compromised or malicious node). **PBFT (Practical Byzantine Fault Tolerance)**, by Castro and Liskov (1999), made this practical. To tolerate f Byzantine nodes it needs $3f+1$ total replicas (e.g. tolerate 1 liar with 4 nodes), because honest nodes must out-vote liars even when liars send conflicting stories. Its normal path has three all-to-all phases — **pre-prepare** (leader proposes order), **prepare** (replicas cross-confirm they saw the same proposal), and **commit** (replicas confirm enough others are ready) — each needing about $2f+1$ matching messages so the honest majority always intersects. This is far more message-heavy than

Raft/Paxos, which is why BFT is reserved for settings where you genuinely don't trust the participants — most famously **blockchains**, where mutually distrusting parties must still agree on one ledger.

Common mistake: Reaching for Byzantine fault tolerance (PBFT, blockchain-style consensus) inside your own data center. If you run all the nodes yourself, they don't lie — they crash. Crash-fault consensus (Raft/Paxos) is the right tool, and it is dramatically cheaper. BFT pays a steep cost (more nodes, more messages, cryptography) to solve a problem you almost certainly don't have internally.

6.5 When do YOU actually need consensus?

Consensus is powerful but expensive, and most application features do not need it. Use this practical ladder:

1. **Do you need it at all?** A single database with one primary already gives you a consistent, ordered source of truth for free. If one Postgres/MySQL primary (with replicas for read scaling or failover) handles your data, you do *not* need to run your own consensus protocol. Most CRUD apps live here happily.
2. **Do you need agreement on a small piece of critical metadata?** Leader election ("who is the active worker?"), distributed locks, feature flags, service config, cluster membership — these genuinely need consensus. **Use an off-the-shelf system: etcd, ZooKeeper, or Consul.** They expose simple primitives (a consistent key-value store, locks, leases) and have absorbed years of bug-fixing.
3. **Do you need a consistent, sharded database across machines/regions?** Use a system that builds consensus in for you — CockroachDB, Spanner, TiKV, etcd — rather than wiring raw Raft yourself.

Best practice: Do NOT roll your own consensus. This is the single most repeated piece of advice in the field. Even the experts who wrote production Paxos (Google's Chubby team) reported that turning the paper into correct, fault-injecting, real code was far harder than expected. Reach for a battle-tested library or service. Writing your own is a multi-year correctness project, not a sprint task.

The cost you are paying. Every committed decision in Raft/Multi-Paxos needs a round trip to a **majority** of nodes. That means: (1) you cannot go faster than the latency to the median node — across regions that's tens to hundreds of milliseconds *per write*; (2) you need an odd number of nodes (3 or 5 typically) so a majority exists; (3) the system keeps working only while a majority is reachable — lose the majority (e.g. 2 of 3 nodes down) and writes correctly *stop* rather than risk split-brain. This is the CP corner of CAP you met earlier, made concrete.

Example: A team wants "only one worker should process the nightly billing job." Tempting wrong answer: invent a custom heartbeat-and-flag scheme in your app database. Right answer: take a lease/lock from etcd or ZooKeeper — `acquire("billing-leader", ttl=15s)` . Whoever holds it runs the job; if that node dies, the lease expires and another node acquires it. You consumed consensus through a one-line API instead of reimplementing leader election (and its dozen subtle failure cases) yourself.

Key takeaway: Multi-Paxos and Raft are two roads to the same destination — an agreed-upon replicated log, committed in one majority round trip in the steady state via a stable leader. Raft trades flexibility for a complete, understandable spec; Paxos is more general but easier to get wrong. In practice you should almost never implement either: pick etcd/ZooKeeper/Consul for coordination, or a consensus-backed database for data, and reserve consensus for the few decisions that truly require everyone to agree.

Common mistakes & misconceptions

Common mistake: Treating "Multi-Paxos" as a single, copy-pasteable algorithm. The paper sketches the leader/log idea but omits the hard practical machinery (election, gap filling, recovery, membership). Real systems had to design and verify all of that themselves.

Common mistake: Thinking Raft is fundamentally faster or safer than Paxos. They were engineered to give identical guarantees and identical common-case performance. The only deliberate difference is understandability.

Common mistake: Re-running Paxos Phase 1 for every command. The whole point of the stable-leader optimization is to run Phase 1 *once* per leadership term and then use only Phase 2 (one round trip) for each subsequent command.

Common mistake: Using Byzantine fault tolerance (PBFT/blockchain consensus) for internal infrastructure. Your own nodes crash, they don't lie — crash-fault consensus (Raft/Paxos) is correct and far cheaper. BFT is for mutually distrusting parties.

Common mistake: Rolling your own consensus library for a production system. Even expert teams found turning the papers into correct, fault-tested code brutally hard. Use a proven system instead.

Common mistake: Putting your high-volume data path through consensus when you don't need to. Consensus costs a majority round trip per write. Most apps need agreement only on a tiny slice of critical metadata (leader, lock, config), not on every request.

Best practices

Best practice: Use an off-the-shelf coordination service (etcd, ZooKeeper, Consul) for leader election, locks, membership, and config. Consume consensus through a simple lease/key-value API rather than reimplementing it.

Best practice: If you only need a consistent ordered source of truth, a single database primary (with replicas) is often enough. Reach for distributed consensus only when one primary genuinely cannot satisfy your availability or geographic needs.

Best practice: Run an odd number of voting nodes (3 or 5). It maximizes fault tolerance per node and avoids ties when forming a majority. Five nodes tolerate two failures; three tolerate one.

Best practice: Budget for the latency of a majority round trip, especially across regions. Keep latency-sensitive, high-throughput work *outside* the consensus path; put only the must-agree decisions inside it.

Best practice: When you do need a consensus-backed database, pick one that builds it in (CockroachDB, Spanner, TiKV, etcd) so the membership-change, recovery, and log-compaction details are handled and tested for you.

Section summary

- **Replicated log + state machine replication** is the goal: every replica applies the same commands in the same order, so they all reach the same state.
- **Multi-Paxos** = run basic Paxos per log slot; the key optimization is electing a **stable leader** so Phase 1 runs once and each command then commits in a single Phase-2 round trip.
- **Raft** matches Multi-Paxos's guarantees and efficiency but is structured for **understandability**: a strong leader, a contiguous append-only log, terms, and just two RPCs (`RequestVote` , `AppendEntries`).
- Raft's `RequestVote` $\approx$ Paxos Phase 1; Raft's `AppendEntries` $\approx$ Multi-Paxos Phase 2 — the same machine, different presentation.

- Fair verdict: **same correctness and speed**; Raft is easier to implement correctly, Paxos is more flexible and easier to get subtly wrong.
- Real systems: **Raft** — etcd, Consul, CockroachDB/TiKV, Kafka KRaft; **Paxos/ZAB** — Chubby, Spanner, ZooKeeper. Most are coordination services or sharded databases.
- Advanced variants: **EPaxos** (leaderless, fast path for commutative commands), **Flexible Paxos** (only Phase-1/Phase-2 quorums must intersect), **PBFT** (tolerates lying nodes with $3f+1$ replicas and three phases — used in blockchains).
- Use crash-fault consensus (Raft/Paxos) internally; reserve Byzantine fault tolerance for mutually distrusting parties.
- Practical guidance: a single DB primary is often enough; for true agreement use etcd/ZooKeeper/Consul; **do not roll your own**.
- The price of consensus is a **majority round trip per decision** and the need for a reachable majority — writes correctly stop rather than risk split-brain.

Glossary of Terms

Acceptor

A Paxos role. Acceptors are the voting members; a value is chosen only when a majority of acceptors accept it. They remember the highest proposal number they have promised and the last value they accepted.

AppendEntries

The Raft RPC the leader sends to followers to replicate new log entries. With an empty entry list it doubles as the *heartbeat* that keeps the leader's authority alive and resets follower election timeouts.

Agreement

The consensus safety property that no two correct nodes ever decide different values. There is one outcome, not several.

Ballot (ballot number)

Paxos's term for a globally ordered, unique proposal number used to rank competing proposals. The Paxos equivalent of Raft's term / epoch.

Byzantine Fault Tolerance (BFT)

Tolerance of nodes that behave arbitrarily or maliciously — lying, sending conflicting messages — not just crashing. Raft and classic Paxos assume *non-Byzantine* (crash-only) faults; BFT needs different, more expensive algorithms.

Candidate

A Raft server state. A follower that times out waiting for a leader becomes a candidate, increments the term, votes for itself, and asks others for votes via RequestVote.

Commit index

The index of the highest log entry known to be committed. Once an entry is committed it is durable and safe to apply to the state machine; this index only moves forward.

Committed entry

A log entry that has been stored on a majority of servers by the current leader. Committed entries will never be lost and will eventually appear, in the same position, on every node.

Consensus

The problem of getting multiple nodes to agree on a single value or a single ordering of events, despite failures and an unreliable network.

Election timeout

The randomized interval a Raft follower waits without hearing from a leader before starting an election. Randomization spreads out timeouts so candidates rarely clash, avoiding repeated split votes.

EPaxos (Egalitarian Paxos)

A leaderless Paxos variant where any node can commit commands; non-conflicting commands commit in one round trip with no fixed leader, improving latency and balance at the cost of complexity.

Epoch

Another name for term/ballot — a monotonically increasing number marking a leadership "reign."
Used in ZAB and many systems to detect and reject stale leaders.

FLP (impossibility result)

Fischer, Lynch & Paterson's theorem: in a fully asynchronous system with even one possible crash, no deterministic algorithm can guarantee consensus is always reached. Real systems sidestep it with timeouts/randomization to get practical (not theoretically perfect) liveness.

Follower

The default, passive Raft state. Followers do nothing on their own — they respond to RequestVote and AppendEntries from candidates and leaders.

Heartbeat

A periodic empty AppendEntries from the leader that says "I'm still here," suppressing follower election timeouts and preserving the single-leader invariant.

Joint consensus

Raft's safe membership-change mechanism. The cluster passes through a transitional configuration that requires majorities from *both* the old and new member sets, so no moment exists where two disjoint majorities could each elect a leader.

Leader

The single Raft server, elected for a term, that handles all client writes and is the sole source of log entries. All changes flow leader → followers.

Leader completeness

A Raft safety property: any entry committed in a given term is present in the logs of all leaders of later terms. The voting rules ensure only an up-to-date server can win, so committed history is never lost.

Learner

A Paxos role that finds out which value was chosen and acts on it, without participating in voting. In real systems a learner often catches up the application/state machine.

Liveness

The guarantee that something good *eventually* happens — e.g. a decision is reached and progress is made. Contrast with safety. FLP limits liveness guarantees under full asynchrony.

Log / log entry

The ordered, append-only sequence of commands each node stores. A log entry holds a command plus the term in which it was created; agreeing on the log is how nodes stay identical.

Log matching

A Raft property: if two logs contain an entry with the same index and term, then the logs are identical in all entries up to that point. This lets a follower's log be repaired by a simple backward consistency check.

Majority (quorum)

More than half the nodes. Because any two majorities must overlap in at least one node, requiring a majority for every decision guarantees decisions can't conflict. See *Quorum*.

Multi-Paxos

Paxos optimized for a continuous stream of decisions: elect a stable leader once, then skip the prepare phase for subsequent slots, so steady-state commits take a single round trip. This is what real "Paxos" deployments use.

PBFT (Practical Byzantine Fault Tolerance)

A landmark BFT consensus protocol tolerating up to f malicious nodes out of $3f+1$, via multiple voting rounds. The basis for many later blockchain/permissioned-ledger algorithms.

Paxos

Lamport's original consensus algorithm. A single run ("Single-Decree Paxos") agrees on one value via two phases (prepare/promise, then accept/accepted) among proposers and acceptors.

Prepare / Promise (Phase 1)

Paxos phase one: a proposer asks acceptors to "promise" not to accept any lower-numbered proposal; acceptors reply with a promise plus any value they've already accepted, so the proposer respects existing choices.

Accept / Accepted (Phase 2)

Paxos phase two: the proposer asks acceptors to accept a specific value; if a majority accept, the value is chosen.

Proposer

A Paxos role that drives the protocol — picks a ballot number and proposes a value, trying to get a majority of acceptors to accept it.

Quorum

The minimum set of nodes whose agreement counts as the cluster's decision — almost always a strict majority. The overlap property of quorums is what makes consensus safe.

Raft

A consensus algorithm designed for understandability. It decomposes consensus into leader election, log replication, and safety, with a strong single leader. Behind etcd, Consul, CockroachDB, and more.

Replicated State Machine (RSM)

The core pattern: identical state machines on each node that, fed the same commands in the same order, stay in lockstep. Consensus is used to agree on that command order.

RequestVote

The Raft RPC a candidate sends to gather votes during an election. A server grants its vote only if it hasn't voted this term and the candidate's log is at least as up-to-date as its own.

Safety

The guarantee that nothing bad *ever* happens — e.g. two nodes never commit different values at the same log position. Consensus protocols hold safety even during partitions; they may pause liveness instead of risking it.

Snapshot

A compacted image of the state machine at a point in time, used to discard old log entries (log compaction) and to fast-forward a lagging or newly added server.

Split vote

An election where no candidate wins a majority because votes were divided, so a new election must run. Randomized election timeouts make repeated split votes rare.

Term

Raft's logical clock: a monotonically increasing number identifying one leadership period. At most one leader exists per term; higher terms always win over lower ones, exposing stale leaders.

Termination

The consensus liveness property that every correct node eventually decides — the algorithm doesn't run forever. Bounded by FLP in fully asynchronous models.

Validity

The consensus property that the value decided must be one actually proposed by some node — you can't agree on a value nobody asked for.

ZAB (ZooKeeper Atomic Broadcast)

The leader-based atomic-broadcast protocol behind Apache ZooKeeper. Like Raft, it uses a single leader and epochs to deliver a totally ordered, replicated log of updates.

Frequently Asked Questions

Q: Raft or Paxos — which should I actually use?

A: For almost any new system, reach for a battle-tested Raft library (etcd's, or Hashicorp's) — Raft was explicitly designed to be understandable and operable, and the ecosystem of tooling is large. Plain Paxos is famously hard to implement correctly; the version people deploy is Multi-Paxos, which ends up structurally close to Raft anyway. Use Paxos only when you're adopting a system already built on it (e.g. Spanner) or you need a specific variant like EPaxos. Either way, don't write your own from scratch — use a proven implementation.

Q: Why a majority and not all nodes?

A: Requiring *all* nodes means a single failure or a single slow node stops all progress — you'd have zero fault tolerance. A majority lets the cluster keep deciding while a minority is down. Safety still holds because any two majorities must share at least one node, and that overlapping node prevents two conflicting decisions from both "winning."

Q: What happens if two leaders exist at once?

A: It can briefly happen — e.g. an old leader is partitioned away and doesn't yet know a new one was elected. It's safe, though, because terms/epochs are increasing numbers. The new leader operates in a higher term; the moment the old leader contacts a node with the newer term, it learns it's stale and steps down. Crucially, the old leader can't commit anything, because committing needs a majority, and the majority has already moved to the new term.

Q: Does consensus survive a network partition?

A: Safety always survives — you will never get split-brain conflicting commits. Availability does not always survive: only the side of the partition that still holds a majority can make progress; the minority side pauses writes until the partition heals. This is the CAP trade-off in action — these systems choose consistency over availability when forced to pick.

Q: How many nodes do I need, and why are 3 or 5 typical?

A: To tolerate f failures you need $2f+1$ nodes. So 3 nodes tolerate 1 failure and 5 tolerate 2 — the sweet spots for most production clusters. 3 is the cheapest cluster that survives any single failure; 5 buys you more resilience and lets you lose a node for maintenance while still tolerating one unplanned failure.

Q: Why odd numbers of nodes?

A: Adding an even-numbered extra node gives you no extra fault tolerance but raises the majority threshold, so it can actually hurt. A 4-node cluster still only tolerates 1 failure (majority is 3) — same as 3 nodes — yet has more machines to fail. Odd sizes give the best resilience per node and avoid tie-prone majorities.

Q: Can't I just use a single database instead?

A: A single database is simpler and consistent, but it's a single point of failure — when it goes down, everything goes down, and a crash can lose recent writes. Consensus exists precisely to replicate that database across machines so it survives failures without serving conflicting data. If you genuinely don't need high availability or durability beyond one box, a single DB is fine — don't add consensus you don't need.

Q: How do Raft and Paxos relate — is Raft just Paxos?

A: They solve the same problem with the same majority-quorum foundation, and Multi-Paxos and Raft converge on a similar "stable leader + replicated log" shape. The differences are in framing: Raft prescribes a single strong leader, a structured log with the log-matching rule, and explicit membership changes, all chosen for clarity. Classic Paxos is more minimal and general but leaves leader election and the multi-decision machinery as exercises, which is why real Paxos deployments reinvent much of what Raft specifies.

Q: What is a replicated state machine and why does every section mention the log?

A: It's the key reduction. If every node runs the same deterministic state machine and applies the exact same commands in the exact same order, all nodes end up identical. So the only thing you need consensus on is the *order of commands* — i.e. the contents of an append-only log. Both Raft and Paxos are, at heart, machines for agreeing on a log.

Q: If FLP proves consensus is impossible, how do real systems work?

A: FLP says no *deterministic* algorithm can *guarantee* termination in a *fully asynchronous* network where a node might crash — it's about a theoretical worst case where you can never tell "crashed" from "slow." Real systems dodge it by adding timeouts and randomness: they assume the network is *usually* timely. They never violate safety; they only risk a temporary stall in pathological conditions, which is an acceptable trade.

Q: When is an entry "committed," and can a committed entry ever be lost?

A: In Raft, an entry is committed once the current leader has replicated it to a majority of servers. After that it's permanent — the voting rules guarantee every future leader already has it, so it's never overwritten or lost. Uncommitted entries (replicated to only a minority) *can* be discarded if leadership changes; that's correct, because no client was ever told they succeeded.

Q: Why does the leader handle all the writes — isn't that a bottleneck?

A: A single leader makes reasoning about ordering trivial and keeps the protocol simple, which is the whole point of Raft. It is a throughput ceiling, yes. Systems scale around it by sharding (many Raft groups, each with its own leader), by batching and pipelining entries, and by serving some reads from followers with leases. Leaderless variants like EPaxos remove the single leader but pay in complexity.

Q: What's the difference between safety and liveness, and which wins?

A: Safety = "nothing bad ever happens" (no two conflicting decisions). Liveness = "something good eventually happens" (a decision is reached). Consensus protocols always prioritize safety: if forced to

choose, they'd rather stop making progress than risk a wrong agreement. That's why a minority partition halts instead of inventing its own answer.

Q: Do Raft and Paxos protect against malicious or buggy nodes?

A: No. They assume the crash-fault (non-Byzantine) model: nodes may stop, slow down, or restart, but they never lie or send conflicting messages on purpose. Defending against malicious participants needs Byzantine Fault Tolerant protocols like PBFT (and the consensus mechanisms behind many blockchains), which require more nodes ($3f+1$) and more communication.

Q: What's a heartbeat and what is an election timeout?

A: The heartbeat is the leader's periodic "I'm alive" message (an empty `AppendEntries`) that stops followers from starting elections. The election timeout is how long a follower waits with no heartbeat before deciding the leader is gone and becoming a candidate. The timeout is randomized per node so that when a leader dies, one follower usually times out first and wins cleanly, avoiding split votes.

Q: How does the cluster change membership without breaking safety?

A: Naively swapping the member list is dangerous — for a moment the old and new sets could each form a majority and elect rival leaders (split-brain). Raft avoids this with *joint consensus*: a transitional configuration where decisions need a majority from both the old and the new sets simultaneously, so there's never a window where two disjoint majorities exist. Once everyone has the joint config, the cluster moves to the new one.

Q: Why do these systems use snapshots?

A: The log grows forever otherwise. A snapshot captures the state machine's current state so all the log entries that produced it can be discarded (log compaction), keeping disk and replay times bounded. Snapshots also let a brand-new or far-behind server catch up instantly by installing the snapshot instead of replaying the whole history.

Revision Cheat Sheet

One-line everything: **Consensus = agree on one ordered log of commands, using a majority quorum, so identical replicated state machines stay in lockstep despite crashes and partitions.**

The Consensus Requirements

- **Agreement** — no two correct nodes decide differently (safety).
- **Validity** — the decided value was actually proposed by someone (safety).
- **Termination** — every correct node eventually decides (liveness).
- **FLP** — in a fully async network with crashes, no deterministic algorithm guarantees all three; real systems use timeouts + randomness to get practical liveness, never sacrificing safety.

Quorum / Majority Rule

- Quorum = strict majority = $\lfloor n/2 \rfloor + 1$.
- Tolerate **f** failures → need **2f+1** nodes (3 tolerate 1, 5 tolerate 2).
- Safety comes from **overlap**: any two majorities share ≥ 1 node, so two conflicting decisions can't both win.
- Use **odd** node counts — an even extra node raises the threshold without adding fault tolerance.

Raft — Server States

State	Role
Follower	Passive; responds to RPCs; starts election if heartbeat times out.
Candidate	Bumps term, votes for self, requests votes; wins on majority.
Leader	Handles all client writes; sends heartbeats; replicates the log.

Raft — The Two RPCs

RPC	Sent by	Purpose	Granted/accepted when
RequestVote	Candidate	Win an election	Voter hasn't voted this term AND candidate's log is $\geq$ as up-to-date.
AppendEntries	Leader	Replicate entries / heartbeat (empty)	Term is current AND prev entry's index+term match (log-matching check).

Raft — Key Properties

- **Election safety** — at most one leader per term.
- **Log matching** — same index+term $\Rightarrow$ logs identical up to that point.
- **Leader completeness** — a committed entry exists in every future leader's log.
- **Committed** = replicated to a majority by the current leader $\Rightarrow$ durable, never lost.
- **Membership** changes via **joint consensus** (old+new majorities both required) to avoid split-brain; **snapshots** compact the log and catch up laggards.

Paxos — Two Phases (Single-Decree)

Phase	Proposer asks	Acceptor does
1: Prepare / Promise	"Promise to ignore ballots < N"	Promises; returns any value already accepted.
2: Accept / Accepted	"Accept value V at ballot N"	Accepts if it hasn't promised a higher N. Majority accept $\Rightarrow$ chosen.

Paxos safety rule: if any acceptor already accepted a value, a new proposer must **re-propose that same value** (the highest-ballot one it learns about in Phase 1) — this is what prevents two different values being chosen.

- Roles: **Proposer** (drives), **Acceptor** (votes), **Learner** (finds out the result).
- **Multi-Paxos**: elect a stable leader once, skip Phase 1 for later slots $\Rightarrow$ steady-state commit = one round trip (just like Raft).

Raft vs Paxos — One-Liners

- Same problem, same majority foundation; both reduce to agreeing on a log.
- Raft = prescriptive, single strong leader, structured log, built for understandability.
- Paxos = minimal, general, leaderless in theory; leaves leader election + multi-decision as DIY.
- Multi-Paxos and Raft converge on "stable leader + replicated log" in practice.
- Term (Raft) = Epoch = Ballot (Paxos) — the increasing number that exposes stale leaders.
- Both are **crash-fault** tolerant only — neither handles Byzantine/malicious nodes (use PBFT for that).

Real-World Systems

System	Algorithm	Used for
etcd	Raft	Kubernetes' config/state store.
Consul	Raft	Service discovery, config, locks.
CockroachDB / TiKV	Raft (per shard)	Distributed SQL/KV replication.
ZooKeeper	ZAB (Raft-like)	Coordination, locks, leader election.
Google Spanner / Chubby	(Multi-)Paxos	Globally-distributed DB; lock service.
PBFT / blockchains	BFT consensus	Agreement among untrusted/malicious nodes.

Rules of Thumb

Use **3 nodes** for "survive one failure," **5** for "survive two / maintenance + a failure." Beyond 7, write throughput suffers — shard into multiple groups instead.

Don't roll your own consensus. Use a proven Raft library (etcd/Hashicorp). Subtle bugs in election or commit logic cause silent data loss that tests rarely catch.

Every decision costs at least one **majority round trip** — budget that latency. Cross-datacenter clusters pay WAN latency on every write; keep the quorum geographically tight or accept the cost.

- Partition ⇒ only the majority side makes progress; the minority pauses (consistency over availability).
- Scale reads with follower reads + leader leases; scale writes by sharding into many consensus groups.
- Cap the log with snapshots; bootstrap new nodes from a snapshot, not full replay.